



VULOC: Vulnerability location framework based on assembly code slicing[☆]

Xinghang Lv^{a, *}, Jianming Fu^{a, *}, Tao Peng^b

^a The Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan, 430000, Hubei, China

^b The School of Computer Science and Artificial Intelligence, Wuhan Textile University, Wuhan, 430000, Hubei, China

ARTICLE INFO

Dataset link: <https://github.com/DataAvailable/VULOC>

Keywords:

Vulnerability location
Assembly slicing
Vulnerability detection
Code blocks
Deep learning

ABSTRACT

Automated detection of software vulnerabilities is an important topic in software security. Although the currently proposed deep learning-based approaches are effective in detecting vulnerabilities, their lack of accuracy in pinpointing the location of vulnerabilities leads to significant limitations in real-world usage. To address the above problem, we propose a vulnerability location framework based on assembly code slicing, VULOC, which achieves high detection capability and localization accuracy. VULOC first compiles C/C++ programs to obtain assembly code containing addresses. Then we use Addr2line to generate the mapping between assembly code and source code line numbers, and slice the assembly code into code blocks, which are encoded into the neural network model. Finally, we propose the BLSTM-LOC model for learning vulnerability features and predicting vulnerability locations. To the best of our knowledge, it is the first time that the mapping relationship between assembly code and source code line numbers is exploited for vulnerability detection. Experimental results show that VULOC exhibits higher performance than the current state-of-the-art vulnerability detection methods, both on existing datasets and real-world software products in vulnerability detection.

1. Introduction

Software vulnerabilities refer to flaws in the software, which can lead to serious consequences when exploited by attackers. Although experts are striving to identify these vulnerabilities, the situation is still not optimistic due to the increase in software complexity. As the first step for software security check, source code vulnerability detection, can effectively detect vulnerabilities the first time, to avoid losses caused by vulnerabilities after subsequent software releases. Therefore, it is necessary to develop a vulnerability detector with a low false-negative rate and high localization accuracy.

The current mainstream vulnerability detection methods are divided into static analysis, dynamic analysis. The static analysis (Secure Software Inc., 2023; Liang et al., 2016; Wheeler, 2023; Ltd., 2023) does not require execution of the program during the analysis of the code, but there is a high false-negative rate (FNR). Dynamic analysis (Manès et al., 2019; Peng et al., 2018; She et al., 2020; Niu et al., 2020) are divided into fuzzing and taint analysis, compared with static analysis to improve the detection accuracy, but requires more resources and relatively low efficiency. To solve the above problems, machine learning (ML) is gradually applied to vulnerability detection. ML is capable of learning complex patterns from code to vulnerabilities and adapting

to various programming styles and code scales. In contrast, traditional static analysis tools rely on manually defined rules and syntax tree analysis, which struggle to generalize to new types of vulnerabilities or unknown patterns (Lipp et al., 2022). Furthermore, ML-based approaches are better equipped to handle contextual information, which is particularly crucial in assembly-level analysis. Recent studies on using ML to learn assembly representations have also confirmed its potential in assembly instruction embedding (such as control flow and data flow embedding) (Yu et al., 2020; Li et al., 2021a) and binary vulnerability mining (Jiang et al., 2024; Yan et al., 2021), with experimental results further validating its effectiveness in real-world scenarios. Additionally, it has shown excellent performance in using ML to learn more advanced source code features. As Li et al. (2022) presented SySeVR to first define four vulnerability syntax features and generated code slices corresponding with the vulnerable statements through control flow (Ferrante et al., 1987) and data flow (Ferrante et al., 1987), and used the Bidirectional Gate Recurrent Unit (BGRU) (Liu et al., 2019a) model to detect vulnerabilities at the program slice level. Lv et al. (2023) proposed a method, BovdGFE, for detecting buffer overflow vulnerabilities in C/C++ programs based on graph feature extraction. It includes the code representation sequence (CoRS) (Lv et al.,

[☆] Editor: Tao Zhang.

* Corresponding author.

E-mail addresses: lvxinghang@whu.edu.cn (X. Lv), jmfu@whu.edu.cn (J. Fu), pt@wtu.edu.cn (T. Peng).

2023), which combines control flow, data dependencies and syntactic structure of vulnerable code to reduce information loss during code representation. Finally, a deep learning model is used to learn vulnerability features on CoRS and perform vulnerability classification at the function level. Grieco et al. (2016) developed and implemented VDiscover, which uses lightweight static and dynamic features to predict whether a testcase is likely to contain software vulnerabilities at the program level. Devign (Zhou et al., 2019) learns a rich set of code semantics for graph-level classification, and it includes a new *Convolutional Neural Network* (CNN) (Li et al., 2021c) module for efficient extraction of useful features in a rich representation of nodes. These works demonstrate that ML can more accurately capture potential vulnerability patterns in code, thereby improving both the efficiency and accuracy of vulnerability detection.

However, most current methods predict vulnerabilities at a coarse-grained level, such as the program, function, and slice levels. That is, their vulnerability detection targets are described as “program $\rightarrow$ program” (whether the program contains potential vulnerabilities), “program $\rightarrow$ function” (whether the function contains potential vulnerabilities), and “program $\rightarrow$ slice” (whether the slice contains potential vulnerabilities). Their detection results are a program, a function or even a few dozen lines of code and cannot be located in the line of source code where the vulnerability is, which leads to limitations and inconveniences in real-world usage. The prediction goal of our study is to learn a mapping from the source program P to the line of code l_i , similar detection goals to LineVul (Fu and Tantithamthavorn, 2022) and IVDetect (Li et al., 2021b), which can be described as follows: given a source program P , predict the lines of code in P that may contain vulnerabilities, denoted $f : P \rightarrow l_i$. The shortcomings of current vulnerability detection methods in terms of insufficient detection granularity or localization accuracy motivate us to investigate a novel vulnerability detector.

1.1. Challenges

To solve the above problems, we designed VULOC, a vulnerability detection method that can accurately predict the location of vulnerabilities. However, there are two challenges in the design of VULOC as follows.

- **Challenge 1.** In vulnerability detection tasks, program slicing³ is often used to extract statements that may be related to vulnerable code, thereby removing unrelated code to enhance the model’s vulnerability detection capability, which is an important preprocessing technique. The program slicing in source code is typically implemented based on the control flow and data flow. Then, how to perform the program slicing at the assembly code level?
- **Challenge 2.** Recent neural network-based vulnerability detection methods (Lv et al., 2023; Li et al., 2022; Zou et al., 2022; Zhou et al., 2019) have demonstrated their strong feature extraction capabilities, enabling them to capture and learn more comprehensive vulnerability semantics and cover a wider range of vulnerability types, thus eliminating the need for manually defined detection rules. However, standard neural network models such as *Recurrent Neural Network* (RNN) (Grossberg, 2013), *Long-Short Term Memory* (LSTM) (Hochreiter, 1997), *Gated Recurrent Unit* (GRU) (Chung et al., 2014), *Graph Neural Network* (GNN) (Scarselli et al., 2008), *Convolutional Neural Network* (CNN) (Wu et al., 2022), etc. can only perform simple classification of vulnerabilities (i.e., their output is “yes” or “no”) and have high false-positive rate (FPR) and false-negative rate (FNR) due to the vulnerability-independent features learned. Thus, how to design a specific neural network model that has the capability to predict the location of vulnerabilities?

1.2. Contributions

In this paper, our contributions are as follows:

- We propose VULOC, a vulnerability location framework based on assembly code slicing, focusing on C/C++ programs vulnerability detection. Unlike most popular vulnerability detection tools that predict vulnerabilities only at the function level or code slice level, we detect vulnerabilities at the line number level. Back to the essence of vulnerability detection, that is, VULOC takes the source programs as input and the line number of the vulnerability as output. In daily work, VULOC can assist software developers in quickly locating vulnerabilities for subsequent patching efforts.
- To enable the neural network model to learn more vulnerability information, we have conducted extensive research on the feature engineering of source code vulnerabilities, resulting in a novel representation structure for vulnerability code. This structure is composed of assembly instructions obtained by disassembling binary files and the corresponding source code line numbers for each assembly instruction. This new representation structure is designed to enhance the capability of vulnerability detection and improve the precision of vulnerability localization.
- We propose an improved *Bi-directional Long-Short Term Memory* (BLSTM) (Schuster and Paliwal, 1997) network model by adding the vulnerability localization layer, named BLSTM-LOC. The localization layer includes the 01 matrix constructed from source code line numbers and vulnerability line numbers, the *K-Max Pooling* layer for learning the top K important features, and the *Global Average Pooling* layer for learning global information.
- Compared with the state-of-the-art methods, VULOC shows higher performance in terms of accuracy (97.8%) and FNR (0.2%). In particular, it achieves an average prediction distance of 0.78 lines (the distance between the predicted and the real vulnerability code line number) and an average output of 1.0 line (the granularity of predicted vulnerability code) of vulnerabilities in vulnerability localization, with extremely high localization accuracy and granularity.

1.3. Roadmap

The rest of this paper is organized as follows. Section 2 introduces the basic ideas and motivations of our paper. Section 3 presents the foundational definitions of this paper. Section 4 provides a detailed description of the VULOC framework. Section 5 shows our experiments and results. Section 6 discusses the limitations of the approach in this paper. Section 7 presents a discussion on the usability of the methodology described in this paper. Section 8 describes the related work. Section 9 is the conclusion of this paper.

2. Basic ideas and motivations

2.1. Basic ideas

The basic idea of VULOC is to transform the source programs into the assembly code and the corresponding addresses, then convert the address into the line number of the source code. These assembly codes are classified as code blocks without vulnerabilities and with vulnerabilities. Each of these code blocks is recorded which source program it belongs to and encoded as vectors for the training and subsequent testing phases of the neural network.

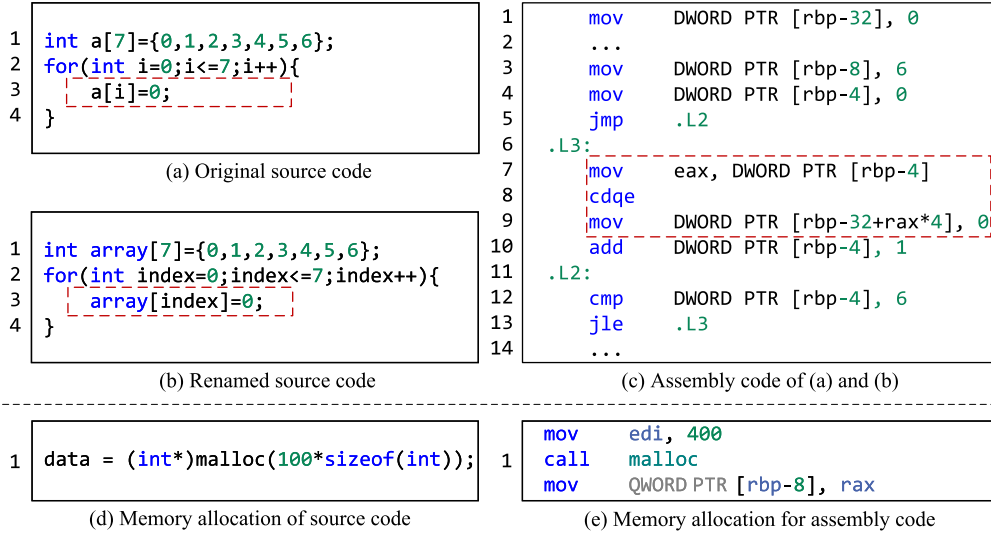


Fig. 1. The motivation for using assembly code. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

2.2. Motivations

The following findings motivated us to use assembly code slicing for vulnerability detection.

Motivation 1. The assembly code is unaffected by variable and parameter renaming, which enhances the robustness of the vulnerability detection model. Rahman et al. (2024) have demonstrated that many source code-based deep vulnerability detection methods, such as Devign (Zhou et al., 2019), Reveal (Chakraborty et al., 2021), CodeBERT (Feng et al., 2020), GraphCodeBERT (Guo et al., 2020), and UnixCoder (Guo et al., 2022), learn spurious features like variable names, parameter names, and API names, rather than the semantic information they represent. We illustrate this with an “array out of bounds” example: (i) From the perspective of source code analysis in Fig. 1(a), the loop’s termination condition is “ $i \leq 7$ ”, which results in an out-of-bounds access when the statement “ $a[7] = 0$ ” is executed in line 3, causing an infinite loop. Typically, baseline methods can learn such simple vulnerability patterns from source code. However, if the variable names in the source code are changed (in real-world projects, variable names are usually more complex; for simplicity, we rename the array “ a ” to “ $array$ ” and the index “ i ” to “ $index$ ” (as Fig. 1(b))), these baselines may fail to predict correctly. (ii) From the perspective of assembly code analysis in Fig. 1(c), in line 9, the value 0 is assigned to “ $[rbp-32 + rax*4]$ ”. Suppose at some point, the value of “ rax ” (the register storing the variable “ i ”) is 7, indicating an index out-of-bounds condition. At this point, “ $[rbp-32 + rax*4] = [rbp-4] = i = 0$ ” (as shown in line 4). This means that whenever “ $i = 7$ ”, the assignment “ $a[i] = 0$ ” in line 3 of the source code will overwrite “ i ” with 0, causing “ i ” to alternate between 7 and 0 within the loop, eventually leading to an infinite loop. After renaming the variables in the source code, we observe that the behavior of the assembly code remains unchanged (as highlighted by the red dashed lines in the figure). This is because array read and write operations in the assembly code are implemented through opcodes and operands, and renaming variables in the source code does not affect these operations. Therefore, our primary motivation for working at the assembly level is to avoid the semantic uncertainty introduced at the source code level due to human-driven refactoring (e.g., arbitrary changes to variable names or non-equivalent adjustments to code structure). The ultimate goal is to enable the model to fully capture the semantic features represented (or encoded) by variables or registers, rather than relying on spurious features such as variable names themselves. This approach is intended to improve the robustness and accuracy of the model to a certain extent.

Motivation 2. The allocated memory size is “implicit” in the source code, whereas it is “explicit” in the assembly code. As shown in Fig. 1(d), deep learning-based vulnerability detection methods that rely on source code need to scan each token from left to right, calculating the values of “ $sizeof(int)$ ” and “ $100* sizeof(int)$ ” before determining that the “ $data$ ” array is allocated 400 bytes of memory for subsequent memory vulnerability prediction. In this process, the allocated memory size is “implicit”. Due to the complexity of the surrounding code context, deep learning models may fail to correctly capture the semantics of “ $100* sizeof(int)$ ” leading to erroneous predictions. However, in the assembly code, the information about the allocation of 400 bytes is explicitly conveyed through the instructions “ $mov\ edi, 400$ ” and “ $call\ malloc$ ” (as shown in Fig. 1(e)). Thus, the allocated memory size is “explicit” in the assembly code, which can assist neural networks in identifying vulnerabilities related to memory operations.

Motivation 3. The vulnerability sink manifests as multiple instructions in the assembly code, whereas it appears as a single line of code in the source code, providing a certain degree of tolerance for vulnerability localization. These assembly instructions are transported into the neural network model as vulnerability features for learning, thus increases the fine-grained level of vulnerability detection at the source. Meanwhile, it is clear which assembly instructions the vulnerable lines in the source code are compiled to. After obtaining the mapping of assembly code to source code line numbers, the assembly code is encoded as vectors and input to the neural network model, and then the predictions of the model on assembly code are mapped to the source code to get the location of the vulnerabilities.

3. Fundamental definition

In this section, we first illustrate a vulnerability instance to help understand what the vulnerability is, followed by presenting the relevant foundational definitions in this paper.

Fig. 2 illustrates an instance of “CWE-415: Double Free Vulnerability”. In particular, to facilitate description and aid in understanding the vulnerability, we have chosen a simple instance for illustration purposes. Here, we highlighted the statements related to vulnerabilities in red, indicating the presence of control or data dependency between these statements. The deeper shade of red in the code signifies a higher degree of relevance to the vulnerability sink statement, which is the detection target of VULOC. In the source code, memory allocation is performed using the function “ $malloc()$ ” on line 7, and the allocated memory is freed using “ $free()$ ” on line 9. However, in the for loop

1	void cwe_415_double_free_char()	
2	{	
3	int i,j;	
4	char * data;	
l_5	5 data = NULL;	s_5
6	for(i = 0; i < 1; i++){	
l_7	7 data = (char *)malloc(100*sizeof(char));	s_7
8	if (data == NULL) {exit(-1);}	
l_9	9 free(data);	s_9
10	}	
11	for(j = 0; j < 1; j++){	
l_{12}	12 free(data);	s_{12}
13	}	
14	}	

Fig. 2. CWE-415: Double free vulnerability. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

statement on line 11, the previously freed memory “data” is freed again on line 12, resulting in a double free vulnerability. The double free vulnerability can lead to issues such as application crashes and denial of service attacks, and is one of the common vulnerabilities in C/C++. In recent years, examples of security flaws caused by the double free vulnerability include CVE-2022-40515, CVE-2021-33304, and CVE-2023-28464, among others.

Typically, a source program consists of many functions, and vulnerabilities are generally leaked in single or multiple function calls. Functions contain the complete syntax and semantic information of the vulnerability, and we slice the source program to obtain the following definition:

Definition 1 (Source code slice-based Feature Blocks or ScFB). Given a c program P , which consists of multiple functions f_n , it can be expressed as $P = \{f_1, f_2, \dots, f_n\}$. Each function f_n is composed of several statements s_m , denoted as $f_n = \{s_1, s_2, \dots, s_m\}$. For statement s_m , we assign a unique number l_m in order, to mark the position of the statement in the program P (i.e., the line number). We slice the function f_n based on the four vulnerability candidates proposed by Li et al. (2022). A ScFB z_n consists of the line number l_m and the statement s_m , denoted as $z_n = \{l_1 : s_1, l_2 : s_2, \dots, l_m : s_m\}$, where $s_1, s_2, \dots, s_m$ are statements with control or data dependencies.

To help understand Definition 1, we use an example to explain a ScFB. As shown in Fig. 2, the statements in lines 5, 7, 9, and 12 constitute a ScFB, meaning that the ScFB is a collection of the line numbers and all code related to the vulnerability sink statement. In this specific case, ScFB is represented as $z_n = \{l_5 : s_5, l_7 : s_7, l_9 : s_9, l_{12} : s_{12}\} = \{5: \text{“data=NULL;”}, 7: \text{“data=(char*)malloc(10*sizeof(char));”}, 9: \text{“free(data);”}, 12: \text{“free(data);”}\}$.

Considering that the full assembly code contains too much redundant information that is not relevant to the vulnerability, we propose to generate the assembly code slice corresponding to ScFB as the representation of the vulnerability source code, leading to the following definition:

Definition 2 (Assembly code slice-based Code Blocks or AcCB). Given a source program P , and the ScFB z_n in P , the assembly code corresponding to program P is represented as P' . AcCB is composed of the assembly instructions corresponding to ScFB, denoted as y_n , which is multi-line assembly instruction $s'_{m,j}$ in assembly code P' , where $(0 \leq j \leq m)$. s'_m is the assembly instruction for the statement s_m in the source program P corresponding to assembly code P' , expressed as

0x0000000000001189	2	endbr64		
0x000000000000118d	2	push %rbp		
...	...	...		
0x0000000000001195	5	mov \$0x0, -0x8(%rbp)		
0x00000000000011a6	7	mov \$0x64, %edi	$s'_{7,1}$	s'_7
0x00000000000011ab	7	callq 0x1080 <malloc@plt>	$s'_{7,2}$	s'_7
0x00000000000011b0	7	mov %rax, -0x8(%rbp)	$s'_{7,3}$	s'_7
0x00000000000011c5	9	mov -0x8(%rbp), %rax	$s'_{9,1}$	s'_9
0x00000000000011c9	9	mov %rax, %rdi	$s'_{9,2}$	s'_9
0x00000000000011cc	9	callq 0x1070 <free@plt>	$s'_{9,3}$	s'_9
0x00000000000011e4	12	mov -0x8(%rbp), %rax	$s'_{12,1}$	
0x00000000000011e8	12	mov %rax, %rdi	$s'_{12,2}$	s'_{12}
0x00000000000011eb	12	callq 0x1070 <free@plt>	$s'_{12,3}$	s'_{12}
...	...	...		
0x00000000000011fc	14	AcCB		
0x00000000000011fd	14	leaveq		
	14	retq		

Fig. 3. The structure of assembly code with addresses and line numbers. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

$s'_m = \{s'_{m,1}, s'_{m,2}, \dots, s'_{m,i}\}$, where $(0 < i < j)$. Process ① in Fig. 5 shows the step of converting statement s_6 into the assembly instruction s'_6 . The AcCB y_n is the set of multiple assembly instructions s'_m in P' , denoted as $y_n = \{al_1 : s'_1, al_2 : s'_2, \dots, al_j : s'_j\}$, where al_j is the line number of the statement s_m in the source program P corresponding to s'_m .

For example, Fig. 3 illustrates an instance of assembly code obtained from the vulnerability function in Fig. 2. The structure of the assembly code is divided into three parts from left to right: instruction addresses, source code line numbers, and assembly instructions. We have highlighted the AcCBs in the assembly code using red boxes. In simple terms, the AcCBs are composed of the assembly instructions and source code line numbers of the ScFB in Fig. 2. By Definition 2, this AcCB is expressed as: $y_n = \{al_5 : s'_5, al_7 : s'_7, al_9 : s'_9, al_{12} : s'_{12}\} = \{5 : s'_{5,1}, 7 : \{s'_{7,1}, s'_{7,2}, s'_{7,3}\}, 9 : \{s'_{9,1}, s'_{9,2}, s'_{9,3}\}, 12 : \{s'_{12,1}, s'_{12,2}, s'_{12,3}\}\} = \{5: \text{“movq } 0 \times 0, -0 \times 8(\%rbp)\text{”}, \dots, 12: \text{“callq } 0 \times 1070 \text{ <free@plt>”}\}$.

Definition 3 (Program Slicing). Given a program P with a point *sink* (such as a statement or variable usage), the slice S is a subset of program P containing all statements relevant to the *sink*. Specifically, the program slice S_{sink} is the set of statements that satisfy the following condition: for every statement $s_i \in P$, if the execution or result of s_i affects the computation of *sink*, then s_i should be included in S_{sink} . In vulnerability detection tasks, the program slice S_{sink} typically refers to the set of statements s_i that are data or control flow dependent on the sensitive api function *sink*, which may lead to a vulnerability. It is represented as $S_{\text{sink}} = \{s_i, s_j \in P \mid \text{DataDep}(s_i, s_{\text{sink}}) \cup \text{CtrlDep}(s_j, s_{\text{sink}})\}$, where $\text{DataDep}(s_i, s_{\text{sink}})$ denotes the data flow dependence between statements s_i and s_{sink} , and $\text{CtrlDep}(s_j, s_{\text{sink}})$ denotes the control flow dependence between statements s_j and s_{sink} . The slice ScFB in Definition 1 is a program slice.

4. Methodology

This section provides a detailed introduction to the implementation of VULOC.

4.1. Overview of VULOC

Fig. 4 depicts the overall architecture of VULOC, which combines assembly code representation and deep learning models on the basis of source programs to build a specific vulnerability detection framework. The input to VULOC is training samples or test samples, both of them containing vulnerable and non-vulnerable programs, and the output is the line number of the vulnerability in the source code. For VULOC to

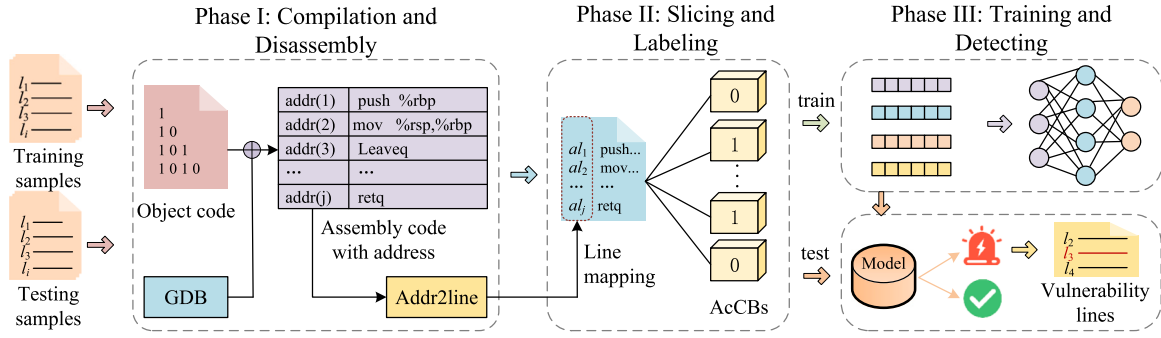


Fig. 4. The framework of VULOC.

work properly, the input data should satisfy the following two principles: (i) the training and testing samples as input should be successfully compiled into the object code and their corresponding assembly code; (ii) the location of the vulnerabilities in each training sample needs to be known for O1 matrix generation. The vulnerability detection can be divided into three phases as follows.

- **Phase I: Compilation and Disassembly.** Compile the source code into the object code (i.e. binary file), and disassemble it for using GDB (Bartlett, 2021) to get the assembly code with the address.
- **Phase II: Slicing and Labeling.** Convert the address in the assembly code to the line number of the source code by the Addr2line (Linux tools, 2019) and slice the assembly code by the line numbers in ScFB to obtain AcCBs, and finally label these as vulnerable or non-vulnerable.
- **Phase III: Training and Detecting.** These AcCBs are encoded as vectors and input to the neural network model for training, and the trained model is used in the testing phase for vulnerability detection.

4.2. Compilation and disassembly

This section provides a comprehensive explanation of compiling and disassembling the source code to obtain the assembly code, along with the corresponding source code line numbers for each assembly instruction.

Since the input of our model is assembly code, it is necessary to construct assembly code that includes the line numbers of the source code to enable the neural network model to predict the vulnerability locations in the source code. Specifically, we utilize the model to output prediction values (discrete values from 0 to 1.0) for each line of assembly instructions in the assembly code. The line number corresponding to the assembly instruction with the highest prediction value is considered the vulnerability statement in the source code. The construction of the assembly code containing the source code line numbers is divided into two steps:

- **Step I:** To preserve the structure of the source code, we generated machine code and assembly instructions for the variables, functions, and other elements defined in the code in the most straightforward manner. This approach facilitates the neural network's learning of vulnerability data flows and simulates the process of converging from the source to the bad sink (as shown in Section 5.6). Therefore, we used the default optimization option (commonly "-O0" in most GCC (GNU, 2023) versions) to compile the source code. Specifically, the source code "source.c" is compiled into object code "source" (containing all training and testing samples) with the command "gcc -g -o source -I ./testcasesupport source.c", where "./testcasesupport" represents the header file path.

- **Step II:** Considering that GDB can directly obtain the assembly code of the target function via the command "disassemble + <function name>", the object code is debugged using GDB. After obtaining the object code "source", the commands "gdb source" and "disassemble" are executed to disassemble the object code, resulting in the structure of "addresses : assembly instructions" for each function (as shown by addr(6) and s'_6 in Fig. 5(b)).

4.3. Slicing and labeling

We constructed the assembly code with line numbers from the source code, representing the mapping relationship between the source code and the assembly code. If we directly use the entire assembly code as the input for vulnerability detection, the neural network would learn many irrelevant features due to the large amount of input data, resulting in a higher FNR. Although it is possible to slice the source code based on control flow and data flow, it is challenging to perform program slicing on assembly code. Therefore, to address Challenge 1, we propose an assembly code slicing technique that slices the complete assembly code based on the line numbers in ScFB, resulting in finer-grained AcCB. The generation of AcCBs involves the following two steps:

- **Step I:** This step describes how to generate the mapping structure between the source code line numbers and the assembly instructions. After obtaining the structure of "addresses : assembly instructions", each memory address in the structure is traversed. The addresses are translated into corresponding source code line numbers using Addr2line: the command "addr2line address -e filepath -f -C -s" is then executed to convert the memory address into the corresponding source code line number (of which "filepath" represents the path to the object code "source"), to obtain the structure of "line number : assembly instruction". Note that we calculate and ensure the correctness of the above line number mapping by using the addresses of the API functions called in the assembly code and the offset addresses of the instructions. For example, as shown by process ② in Fig. 5, the addresses "0 × 00000000000000765 - 0 × 00000000000000787" in Fig. 5(b) are mapped to the line number "6" in Fig. 5(c).
- **Step II:** This step describes how to slice the structure obtained in step I to obtain AcCBs. For this purpose, we extract all the functions from the source program. When a function is a user-defined function (as vulnerabilities often occur in user-defined functions), we extract line numbers and statements that have control dependency or data dependency with the four vulnerability candidates proposed by Li et al. (2022), which together constitute the ScFB. Next, we extract the line numbers corresponding to each line of code in ScFB, which collectively form the set l_k and the line numbers al_j of S generated in Step I. When al_j belongs to a subset of l_k , the assembly instruction s'_j corresponding to al_j is extracted. Finally, all the line numbers al_j from set S and the corresponding assembly instructions s'_j collectively form AcCB (as shown in process ③ of Fig. 5).

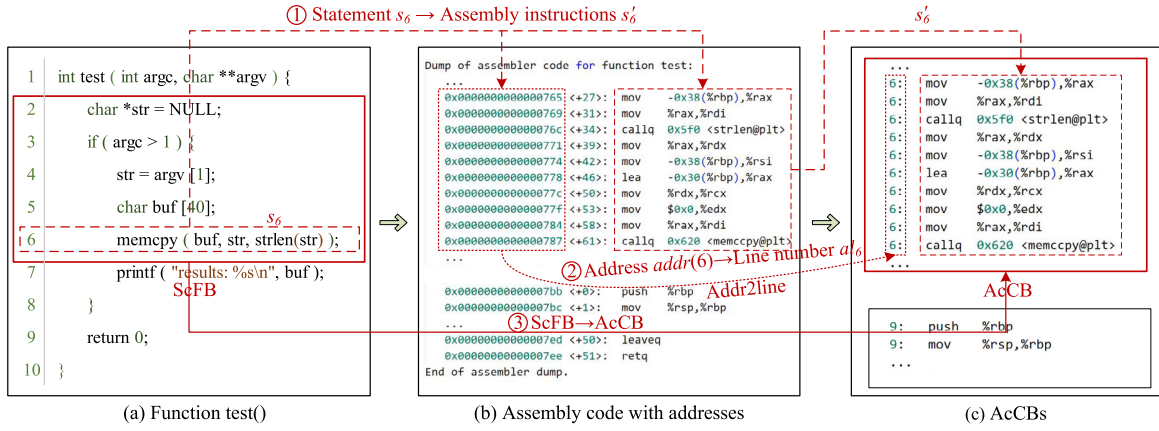


Fig. 5. An example of the ScFB and its corresponding AcCB.

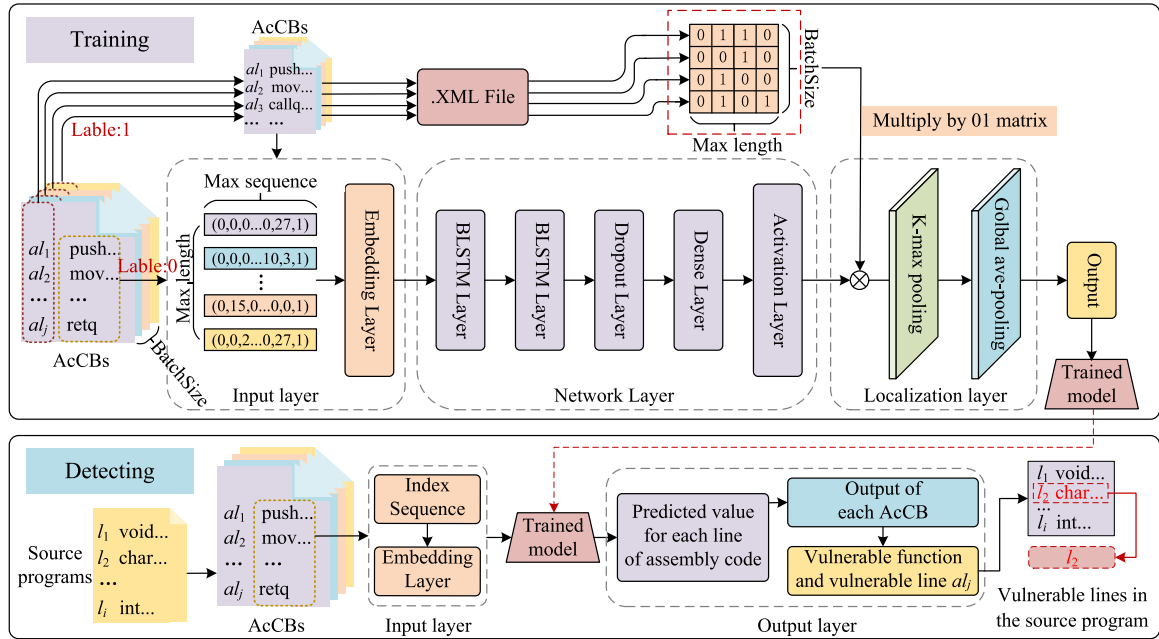


Fig. 6. The architecture of BLSTM-LOC.

Since our research falls under the category of supervised learning, it requires labeling the data to be input. Vulnerability detection can be regarded as a binary classification process, thus we label the AcCBs according to the following principles: query the description file provided with the dataset (i.e., the XML file) and label the AcCB as “1” if it contains a known vulnerability and “0” if the opposite is true. To better help understand, let us take an example to illustrate. Suppose that the set of source code line numbers corresponding to each line of assembly code in the AcCB is denoted as $Cb = \{a_1, a_2, \dots, a_n\} (a_n \in I)$, and we set v as the real vulnerability line of the training sample, and this AcCB is labeled as “1” if $v \in Cb$ and “0” if vice versa.

4.4. Training and detecting

To address the problem that standard neural network models cannot accurately predict the location of vulnerabilities, i.e. **Challenge 2**, we propose the specific deep learning model BLSTM-LOC.

4.4.1. Model building

As shown in the training part of Fig. 6, the BLSTM-LOC model is divided into two parts, training and testing, where the training part

can be further divided into following three components: Input layer, BLSTM layer, and Localization layer.

Input layer. Each labeled AcCB needs to be encoded as a vector before being input to the neural network. For easy understanding, we assume that there is the assembly code “movq \$0 × 0, −0 × 70 (%rbp)”, which is processed into “movq”, “\$0 × 0”, “,”, “−0 × 70”, “(”, “%rbp”, “)” according to the word separation method (as NLTK (Hardeniya et al., 2016)), and encoded into vectors for each AcCB. Since the vector lengths encoded from each AcCB are different, and the neural network requires fixed-length vectors as input, the vectors need to be of uniform length. Suppose the length of the vectors input to the neural network is denoted as δ (i.e., “Max length” in Fig. 6), if the length of the vectors is less than δ , the vectors would be filled with 0 at the end, and if the length of the vectors is greater than δ , the part of the vectors greater than δ would be cropped from the end.

We provide an example using the AcCB from Fig. 3 to demonstrate the process. Firstly, we construct an instruction dictionary (as Fig. 7(b)) by extracting all assembly instructions from the training dataset. These instructions are then sorted in descending order based on their frequencies. Subsequently, we assign index to the instructions in a sequential manner. It is important to note that the index value “0” does not carry specific significance and is solely utilized for padding

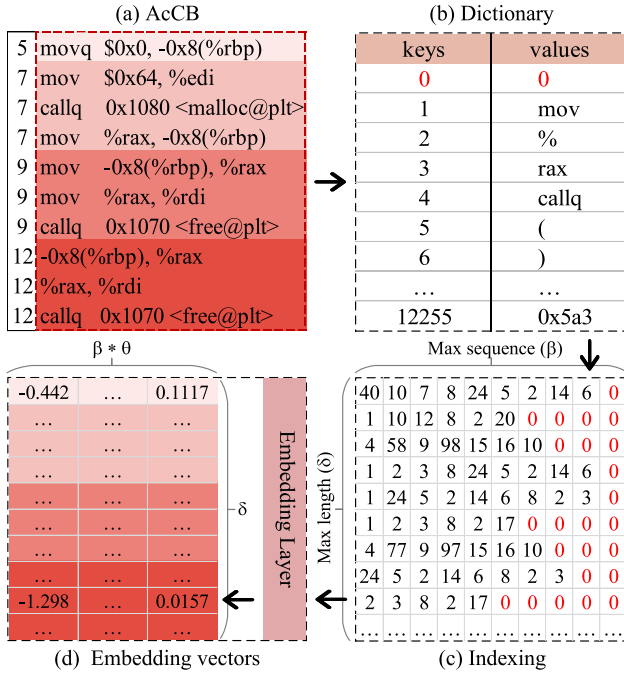


Fig. 7. Input layer.

assembly statements whose lengths are smaller than β , where β denotes the maximum number of assembly instructions per line. Then, the assembly instructions from an AcCB are fed into the input layer (note that the line numbers in the AcCB are not included in the input layer). The instruction dictionary is queried, and the assembly instructions in the AcCB are converted into indices, resulting in the index matrix shown in Fig. 7(c). In the index matrix, each row of numerical sequences uniquely represents a line of assembly code, with the columns indicating the max length (δ) of the AcCB. Finally, the index matrix is passed into the embedding layer, resulting in the embedding vectors corresponding to the AcCB. In this paper, each token is encoded as the vector of length θ , and each AcCB is encoded as the vector of length “ $\delta \times \theta \times \beta$ ” (as depicted in Fig. 7(d)).

Network layer. The fixed-length vectors are encoded and input to this layer for learning vulnerability features, which consists of two standard BLSTM layers (the first BLSTM layer is used to learn generalized features and the second BLSTM layer is used to learn deeper semantic information, similar to Glove (Pennington et al., 2014)), a dropout layer, a dense layer, and an activation layer, where the dropout layer is used to prevent overfitting of the neural network, the dense layer and activation layer are designed to achieve prediction results.

Localization layer. The purposes for us to integrate the localization layer are twofold: (i) the standard BLSTM model learns many features that are not related to vulnerabilities as it treat each node exactly equally (since the initial weight of each neuron in the network is the same), but in the real vulnerability detection, the vulnerable code (or nodes in the neural network) should receive more focus by the neural network, i.e., the vulnerable code nodes should be assigned greater weights. (ii) To maximize the precision of vulnerability detection and accurately predict the key statements leading to vulnerabilities, it is necessary to filter out irrelevant features in the localization layer. This allows the model to fully learn the semantics related to vulnerabilities.

It is worthwhile to represent the output of the BLSTM layer corresponding to the AcCBs of a BatchSize as

$$\gamma = [\gamma_1 \ \gamma_2 \ \dots \ \gamma_{\mu-1} \ \gamma_{\mu}]^T \quad (1)$$

where $1 \leq \mu \leq \text{BatchSize}$ and γ_{μ} is represented as

$$\gamma_{\mu} = [\varphi_{\mu 1} \ \varphi_{\mu 2} \ \dots \ \varphi_{\mu(\delta-1)} \ \varphi_{\mu \delta}]^T \quad (2)$$

where $\varphi_{\mu \delta}$ means the output of each line of assembly code in the AcCB, and δ represents the length of the input vectors (i.e., the “Max length” in Fig. 6). In order to give more attention to the nodes with vulnerabilities, the output of the BLSTM layer is multiplied by O1 Matrix (denoted as M), which is used to retain the information of nodes with vulnerabilities while removing the information of nodes that are irrelevant to the vulnerabilities, M can be expressed by

$$M = \begin{bmatrix} m_{11} & \dots & m_{1\delta} \\ \vdots & \ddots & \vdots \\ m_{\mu 1} & \dots & m_{\mu \delta} \end{bmatrix} \quad (3)$$

where $m_{\mu \delta}$ is calculated by the following principle: the line numbers of the source code corresponding to the known AcCB are denoted by the set $Cb_{\mu} = \{a_1, a_2, \dots, a_n\} (1 \leq n \leq \delta, a_n \in I)$, the real vulnerability line number is known as v . For the vulnerable AcCB, if $a_n = v$, let $m_{\mu n} = 1$, and vice versa, let $m_{\mu n} = 0$.

We utilize Fig. 8 to provide a detailed description of the steps involved in generating the O1 matrix. Fig. 8 illustrates a batch of AcCBs, and for descriptive purposes, we take AcCB 1 as an example for illustration. Firstly, we extract all the line numbers in AcCB 1. For the juliet test suite dataset, we query the “Manifest.xml” file (which describes the vulnerability sink statement and vulnerability types of all “.c/cpp” files in the dataset), and locate the vulnerability sink statement of AcCB 1 according to the program name (we marked which source program it came from when we generated AcCB), i.e., line 12; For the NVD dataset, we use the location of code differences in the *diff* file as the vulnerability line. Finally, following the principle of assigning “1” to vulnerable sink statements and “0” to non-vulnerable sink statement, we construct the O1 sequence corresponding to AcCB 1. For instance, the O1 sequence of AcCB 1 is “0 0 0 0 0 0 0 1 1 1” (referring to the first row of the O1 matrix in Fig. 8). Repeat the above operation to finally obtain the O1 matrix corresponding to a batch of AcCBs, and the size of each O1 matrix is $\delta \times N$, with N denotes the batch size.

After obtaining the O1 matrix M , we multiply it by the output γ_{μ} of the BLSTM layer. Then, we apply a *K-Max Pooling* layer to select the top K important values of the output (i.e., the top K important information predicted by the model). These values are then input into a *Global Average Pooling* layer to compute the average value, resulting in the output ω , which is defined as

$$\omega = \text{Average}(\text{Max}_k(\gamma_{\mu} M)) = (t_1, t_2 \dots t_{\mu-1}, t_{\mu}) \quad (4)$$

The function Max_k calculates the top k largest values of the prediction, the function *Average* is used to calculate the average of the k largest values, and t_{μ} means the output corresponding to each AcCB.

4.4.2. Model training

First, we compile all source programs in the dataset and generate labeled AcCBs using the methods described in Phases I and II. These AcCBs, along with their corresponding labels, are then input to train the model. For vulnerable AcCBs (labeled as “1” in Fig. 6), we extract the line numbers and generate the corresponding O1 matrix. The assembly instructions in the AcCBs are fed into the input layer and embedded as a vector matrix, which serves as the vulnerability feature input into the BLSTM model. The output is obtained through dropout, dense, and activation layers. The output is then multiplied with the O1 matrix, and the resulting values are fed into the k-max pooling layer and global average pooling layer to select the maximum average value. For non-vulnerable AcCBs (labeled as “0” in Fig. 6), we do not generate the O1 matrix but directly input them into the model to get the output values. The cross-entropy loss is computed for each output value compared to the corresponding label, and this loss is used to update the network model through back propagation during training. Ultimately, a deep learning model BLSTM-LOC for vulnerability detection and localization is finally obtained.

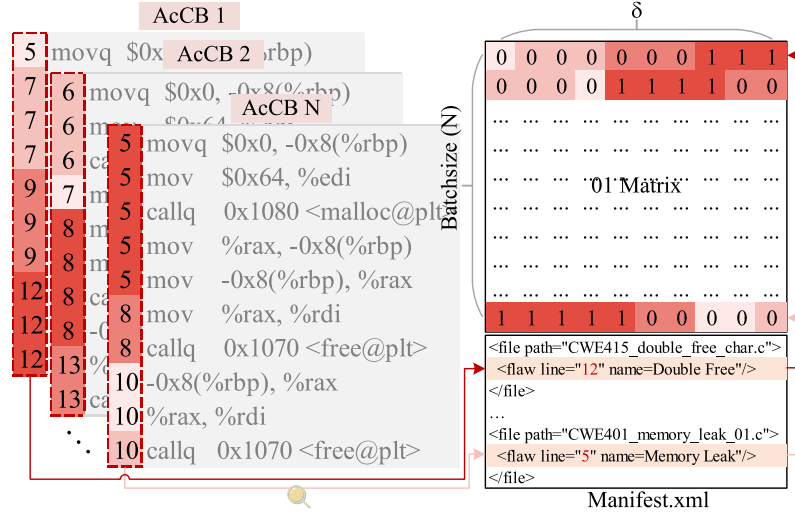


Fig. 8. Generating the 01 matrix.

4.4.3. Detecting vulnerabilities

In order for the neural network to work properly, the generated AcCBs of testing set need to be encoded into vectors as well. Each AcCB is encoded as a vector of length $\delta \times \theta \times \beta = 60,000$, where the values of δ , θ , and β are 120, 50, and 10, respectively, which means that each token is encoded as a vector of length 50 and the max number of assembly instructions per line is 10.

As shown in the detecting part of Fig. 6, to predict the location of vulnerabilities, the detection model takes the source programs as input and generates the AcCBs corresponding to these source programs, the vector encoded from each of them is input into the trained model to obtain the predicted value of each line of assembly code. When the average prediction value of all assembly code lines in the AcCB is greater than the threshold λ (set to 0.5, vulnerability detection and localization constitute a binary classification task, and the threshold of 0.5 is chosen to provide a relatively balanced decision boundary), we assume that this AcCB contains vulnerabilities and output the line al_j with the largest prediction value to finally obtain the vulnerable line of the source program. That is, the BLSTM-LOC model outputs a probability value between 0 and 1, when the probability of a particular sample being input into the model is greater than 0.5, it is considered a positive class (i.e., vulnerable); when the probability is less than 0.5, it is considered a negative class (i.e., non-vulnerable). Note that the line number al_j in the AcCB is only used to map the vulnerability location of the source code in the output phase, and is not fed as a parameter into the trained network model for vulnerability prediction. Therefore, the vulnerability detection step does not require the involvement of the “XML” file, i.e. we do not need to generate the 01 matrix when using the trained BLSTM-LOC model for vulnerability detection, which also fits the realistic vulnerability detection scenario, as the 01 matrix structure does not exist in the real source program.

We use Fig. 9 to help further understand the vulnerability detection results. Fig. 9 shows an example of the output corresponding to a AcCB. “Value” indicates the prediction value of VULOC for each line of the statement, which ranges from 0 to 1.0, the closer to 1.0 the statement indicates the higher possibility of the existence of a vulnerability; “Ground truth” indicates the true label of the statement, where “1” indicates the existence of a vulnerability, and “0” indicates the absence of a vulnerability. We identify statements with a prediction value greater than 0.5 as vulnerable statements, i.e., the vulnerable line in the source program is line 6.

Line	Assembly instruction	Value	Ground truth
4	push %rbp	0.1	0
4	mov %rdi,-0x38(%rbp)	0.1	0
4	mov %fs:0x28,%rax	0.2	0
...	...	0.2	0
5	mov %rax,-0x8(%rbp)	0.3	0
5	xor %eax,%eax	0.4	0
...	...	0.5	0
6	mov -0x38(%rbp),%rax	0.6	1
6	mov %rax,%rdi	0.7	1
6	callq 0x5f0 <strlen@plt>	0.8	1
6	mov %rax,%rdx	0.9	1
6	mov -0x38(%rbp),%rsi	1.0	1
6	lea -0x30(%rbp),%rax	0.9	1
6	mov \$0x0,%edx	0.8	1
6	mov %rax,%rdi	0.7	1
6	callq 0x620 <memcpy@plt>	0.6	1
7	lea -0x30(%rbp),%rax	0.5	0
7	mov %rax,%rsi	0.4	0
7	lea 0xda(%rip),%rdi	0.3	0
...	...	0.2	0
8	je 0x7b9 <test+111>	0.2	0
8	leaveq	0.1	0
8	retq	0.1	0

Fig. 9. An example of the output corresponding to a AcCB.

5. Experiments results and discussion

The goal of the experiments in this paper is to build a vulnerability detector that can be applied to real-world projects and accurately locate the vulnerable line (i.e., the vulnerable sink statement) of the target program while achieving lower FPR and FNR. Consequently, our experiments are designed to answer the following five Research Questions (RQ).

- RQ1: Adding a localization layer to the standard neural network model helps improve vulnerability detection?
- RQ2: In the *K-Max Pooling* layer, what value of *K* makes the model have the optimal localization capability, i.e., what is the optimal *K* value?
- RQ3: Whether assembly code-based vulnerability detection is better than source code-based vulnerability detection?
- RQ4: Compared with the state-of-the-art vulnerability detectors, can VULOC perform better in Vulnerability detection capability?
- RQ5: How well does VULOC detect real-world software source code?

5.1. Experiments details

We used Pytorch 1.9.1 to implement our model, along with GCC 9.4.0, GDB 8.1.1, and Addr2line 2.30 to compile the source program, disassemble the target code, and map addresses to line numbers, respectively. All our experiments were run on a server with the Tesla V100 GPU and Intel Xeon Silver 4214 CPU on Ubuntu 20.04. We perform 10-fold cross-validation to obtain the final trained model.

5.2. Evaluation metrics

The following standard metrics are used to evaluate the vulnerability detection capability of VULOC: False-positive rate (FPR), False-negative rate (FNR), Accuracy (Acc), Precision (Pr), Recall (Re), F1-measure (F1), with their respective calculation formulas provided in Eqs. (5) to (10) where TP denotes the number of samples predicted to be vulnerable and the vulnerability is real, FP denotes the number of samples predicted to be vulnerable but the vulnerability is not existing, TN denotes the number of samples predicted to be non-vulnerable and the vulnerability is not existing, and FN denotes the number of samples predicted to be non-vulnerable but the vulnerability is real.

$$FPR = \frac{FP}{FP + TN} \quad (5)$$

$$FNR = \frac{FN}{TP + FN} \quad (6)$$

$$Acc = \frac{TP + TN}{TP + FP + TN + FN} \quad (7)$$

$$Pr = \frac{TP}{TP + FP} \quad (8)$$

$$Re = \frac{TP}{TP + FN} \quad (9)$$

$$F1 = \frac{2 \cdot Pr \cdot Re}{Pr + Re} \quad (10)$$

To evaluate the vulnerability localization capability of VULOC, we propose *Average Distance* (AD) and average number of output *Vulnerable Lines* (VL), shown in Eqs. (11) and (12), respectively, where l_1 indicates the predicted line number of vulnerability, l_2 indicates the true line number of vulnerability in each sample, n indicates the number of test samples and o_i indicates the number of output vulnerable lines. In real-world vulnerability detection, the ideal state is that the vulnerability detector predicts the same vulnerability line as the real vulnerability line (i.e., $l_1 = l_2$), thus, we can know that when AD tends to be closer to 0, it means that the vulnerability localization accuracy is higher. For the metric VL, the closer its value tends to 1.0, the higher the granularity of the vulnerability detection results.

$$AD = \frac{\sum_{i=0}^n d_i}{n} \quad (11)$$

$$VL = \frac{\sum_{i=0}^n o_i}{n} \quad (12)$$

Table 1

Details of the two types of datasets, where Non-VCBs is the number of non-vulnerable AcCBs, and VCBs denotes the number of vulnerable AcCBs.

Dataset	Programs	Non-VCBs	VCBs	CBs
Juliet Test Suite	17,997	107,068	16,927	123,995
NVD and SARD	4729	83,877	5931	89,808
Total	22,726	190,945	22,858	213,803

5.3. Datasets

In this paper, we collected two types of datasets for our experiments (see Appendix for detailed information).

Juliet Test Suite for C/C++ 1.3 (Center, 2019). A collection of vulnerable datasets created by the National Institute of Standards and Technology (NIST), which includes a total of 64,099 C/C++ files, and it contains 118 types of vulnerabilities, where each vulnerability is uniquely labeled by a Common Weakness Enumeration (CWE) ID number. CWE is a vulnerability identifier, which was proposed by a strategic software security project funded by the National Computer Security Division of the U.S. Department of Homeland Security, and has since become the dominant way of numbering vulnerability types, such as CWE-122 for heap overflow-based buffer overflow vulnerabilities. After sifting out source programs that failed to compile, we selected a total of 17,997 source programs (these programs successfully passed compilation and the corresponding assembly code was obtained) from this dataset (as shown in Table 1), while obtaining the same number of object codes. These object codes are processed by GDB and Addr2line to get assembly codes with line numbers. We sliced these assembly codes and finally obtained 123,995 CBs, of which the number of VCBs is 16,927 and the number of Non-VCBs is 107,068. Each AcCB was marked with which source program it belonged to so that we can subsequently locate the vulnerability. 80% of the CBs were randomly selected as training samples and the remaining 20% as validation samples and test samples. All CBs were encoded as vectors and input to the neural network for training.

NVD and SARD. We collected this dataset from following two sources: The National Vulnerability Database (NVD) (US, 2023) and the Software Assurance Reference Dataset (SARD) (Samate, 2023). For NVD, we collected 1903 real-world programs from three popular open source projects (i.e., Linux Kernel, Wireshark, FFmpeg). We mark the difference between the pre-patched (i.e., programs that have been disclosed as containing vulnerabilities) and patched programs (i.e., programs whose vulnerabilities have been patched) on the NVD as the location of the vulnerability in that program from their *diff* files. The *diff* file records the differences between the vulnerable code in those programs before and after the patch, such as changing the definitions of certain variables, adding new judgment statements, or deleting certain statements. For SARD, we collected 2826 source programs containing synthetic and academic programs, these programs and the functions within them are marked as either non-vulnerable programs/functions or vulnerable programs/functions. As a result, after filtering out the C/C++ programs that cannot be compiled, we collected a total of 4729 source programs (see Table 1 for details), which were processed into 89,808 CBs, 5931 of which were vulnerable. In a similar way, we randomly select 80% of those CBs as training samples and the remaining 20% as validation and testing samples.

The vulnerabilities contained in the above dataset are subdivided into 18 different CWE-IDs and CWE-Names, and 7 different vulnerability types. These vulnerabilities include buffer overflow, integer overflow, memory leak, use after free, uninitialized variable, null pointer dereference, and dead code. The details are shown in Table 2.

Table 2
Vulnerability types and descriptions involved in this paper.

CWE-ID	Vulnerability types	CWE-Name
CWE-119	Buffer overflow	Improper restriction of operations within the bounds of a memory buffer
CWE-121		Stack-based buffer overflow
CWE-122		Heap-based Buffer overflow
CWE-124		Buffer underwrite ('Buffer Underflow')
CWE-125		Out-of-bounds Read
CWE-126		Buffer Over-read
CWE-127		Buffer Under-read
CWE-190	Integer overflow	Integer overflow or Wraparound
CWE-191		Integer underflow (Wrap or Wraparound)
CWE-401	Memory leak	Missing release of memory after effective lifetime
CWE-415	Use after free	Double free
CWE-416		Use after free
CWE-457	Uninitialized variable	Use of uninitialized variable
CWE-824		Access of uninitialized pointer
CWE-476	Null pointer dereference	NULL pointer dereference
CWE-690		Unchecked return value to NULL pointer dereference
CWE-561	Junk code	Dead code
CWE-563		Assignment to variable without use

Table 3
Comparison of different models in detection (unit: %).

Model	FPR	FNR	Acc	Pr	Re	F1
RNN	10.2	8.7	88.1	83.5	90.0	86.6
RNN-LOC	8.0	5.4	92.7	88.7	94.6	91.6
LSTM	7.7	8.1	91.3	90.2	90.7	90.4
LSTM-LOC	4.6	7.3	95.4	93.1	92.7	92.9
GRU	3.5	2.7	96.7	95.4	97.3	96.3
GRU-LOC	2.4	2.6	97.1	96.4	97.4	96.9
BGRU	2.7	2.1	97.6	96.1	97.9	97.0
BGRU-LOC	1.8	0.9	97.6	97.3	97.9	97.6
BLSTM	2.0	1.9	96.5	96.2	97.8	97.0
BLSTM-LOC	1.7	0.2	97.8	97.4	98.0	97.7

5.4. Experiments for answering RQ1

In order to verify the effectiveness of adding a localization layer to the standard neural networks, we construct experiments to compare RNN-LOC, LSTM-LOC, GRU-LOC, BGRU-LOC, and BLSTM-LOC models with the standard neural network models RNN, LSTM, GRU, BGRU, and BLSTM based on assembly AcCBs, respectively. In this experiment, we follow the principles: (i) the same training set; (ii) the same hyperparameters, such as learning rate, batch size, dropout rate, etc.; (iii) only the localization layer is different in the structure of different neural network models (that is, the BLSTM-LOC is obtained by adding a localization layer on the end of the fundamental BLSTM), and the others are consistent. At the same time, to avoid different detection accuracy due to the selection of training and test sets, we use the 10-fold cross-validation (Wong and Yeh, 2019) to train the models and obtain the ultimate experimental results.

Table 3 presents the experimental results. We observe that the models with the added localization layer exhibit better results than the standard models, including a 0.3% improvement in *FPR*, a 1.7% improvement in *FNR*, a 1.3% improvement in *Acc*, a 1.2% improvement in *Pr*, a 0.2% improvement in *Re*, and a 0.7% improvement in *F1* (e.g., BLSTM and BLSTM-LOC). This can be attributed to the advantage of adding the localization layer: in this layer, by multiplying the output of the standard model with the generated 01 matrix, the features that are not relevant to the vulnerability are filtered out and the features that are relevant to the vulnerability are retained, so the neural network learns more interesting information (features relevant to the vulnerability), which leads to higher accuracy and lower *FNR*.

Since the number of positive samples (non-vulnerable) is greater than the number of negative samples (vulnerable) in the dataset and

Table 4
NearMiss and SMOTE algorithms are included to verify the effectiveness in unbalanced data (unit: %).

Algorithms	FPR	FNR	Acc	Pr	Re	F1
None	1.8	0.5	97.6	97.3	97.8	97.5
NearMiss	21.4	10.9	79.2	55.1	60.3	57.6
SMOTE	1.7	0.2	97.8	97.4	98.0	97.7

the same is true in real-world software products. Therefore, we handle this unbalanced data with SMOTE (Chawla et al., 2002) and NearMiss (Mani and Zhang, 2003) algorithms, respectively. Table 4 demonstrates the detection capability of the model with and without imbalanced data processing. We observe that the *Acc* and *F1* are 18.45% and 39.9% lower with NearMiss algorithm processing compared to without the imbalanced data processing ("None"), which indicates that the undersampling technique in NearMiss decreases the training samples and leads the model to possess a poorer detection capability. For the experimental results obtained with SMOTE processing, it is observed that there is a 0.2% improvement in *F1* and 0.3% improvement in *FNR*, which improves rarely, probably due to the fact that the ratio of non-vulnerable and vulnerable samples in the dataset we used is not so disparate (about 6:1).

Finding 1. The addition of a localization layer can effectively improve the vulnerability detection capability of the standard neural network model.

5.5. Experiments for answering RQ2

In this experiment, we seek to find the optimal *K* to provide the model with the best vulnerability localization capability. We select the model BLSTM-LOC with the best performance from RQ1, and then assign *K* to 1, 2, ..., 15, as well as observe the vulnerability detection and localization capabilities of the models under different *K* values.

From Table 5, we can find that when *K* = 6, the model achieves optimal detection and localization performance. At this time, the *ACC* value of the model reaches the highest value of 97.8% and owns the lowest false-negative rate (i.e., 0.2) and average distance (i.e., 0.78). When *K* is greater or less than 6, the performance starts to decrease. Note that this can be explained by the fact that in the AcCB, a line of source code is compiled into 6 lines of assembly code on average, so the first 6 most important pieces of information are selected for learning in the k-max pooling layer, which also filters most of the information that is not related to vulnerabilities and enables model to achieve higher performance. In order to visualize the impact of changes in *K* values on

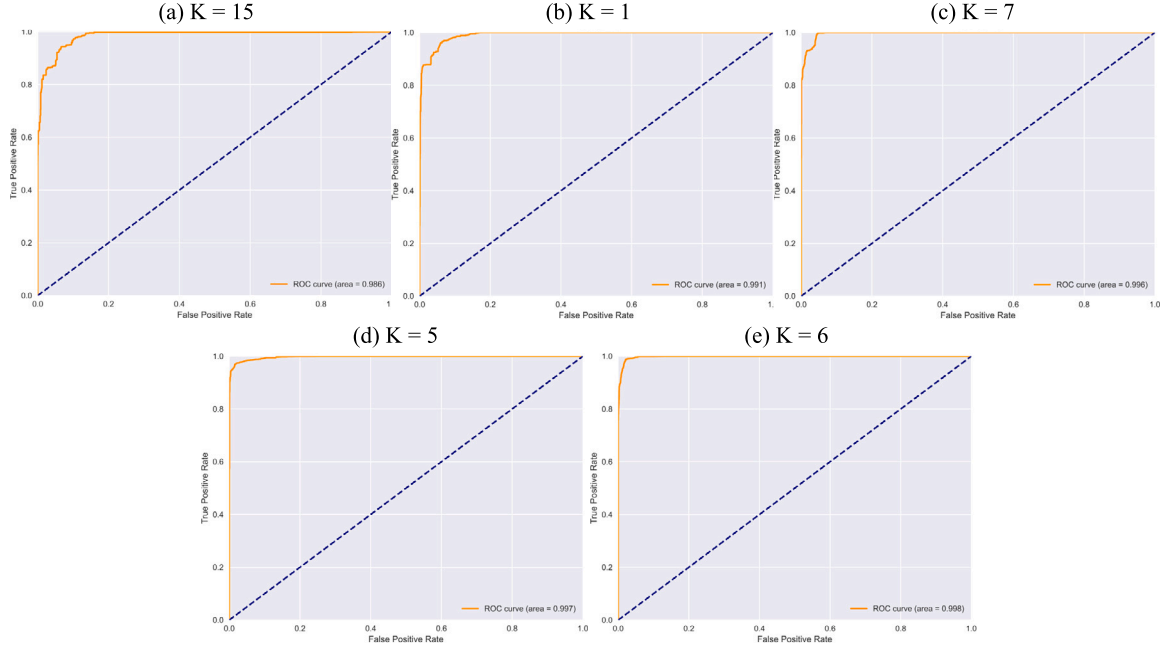


Fig. 10. ROC curves for different values of K.

Table 5

Experimental results obtained from the BLSTM-LOC model at different values of K.

K	FPR (%)	FNR (%)	Acc (%)	Pr (%)	Re (%)	F1 (%)	AD (line)
1	5.2	4.7	94.1	94.3	95.0	94.6	1.98
...	...	...	...	...	...	...	...
5	1.9	0.7	97.5	97.2	97.9	97.5	0.89
6	1.7	0.2	97.8	97.4	98.0	97.7	0.78
7	2.1	1.0	97.3	97.0	97.6	97.3	0.97
...	...	...	...	...	...	...	...
15	5.8	5.4	93.3	94.0	95.1	94.5	2.12

vulnerability detection capabilities, we plotted the ROC curves (short for Receiver Operating Characteristic curves) for the detection results of the models trained at different K values, as shown in Fig. 10. The ROC curves are graphical representations used to assess and compare the performance of binary classifiers. In the ROC curve, the horizontal axis represents the FPR and the vertical axis indicates the true-positive rate (TPR). The area under the ROC curve refers to the AUC value, and its value is closer to 1.0 indicating the higher performance of the model. That is, a better classifier with a higher TPR and a lower FPR will have a curve that is closer to the top-left corner of the plot (as shown in Fig. 10(e) in the paper). The closer the curve is to the diagonal line (representing random guessing), the worse the classifier performs. It is seen that the model reaches its highest AUC value of 0.998 when $K = 6$.

Finding 2. When K takes a value of 6, the model achieves optimal detection and localization performance.

5.6. Experiments for answering RQ3

To answer RQ3, we first conducted ablation experiments using four different optimization levels (O0, O1, O2, and O3) to examine their impact on vulnerability detection performance. Next, based on the findings from Research Questions 1 and 2, we selected the best-performing model (i.e., BLSTM-LOC with $K = 6$ at O0). Finally, using the same dataset (Juliet Test Suite for C/C++ 1.3), we performed comparative experiments on source code-based and assembly code-based representations.

According to Fig. 11, the performance metrics of the vulnerability detection tool (accuracy, precision, recall, and F1 score) exhibit

a consistent downward trend as the compilation optimization level increases from O0 to O3. Specifically, all metrics decline with higher optimization levels. For example, Acc decreases from 97.8% at O0 to 91.8% at O3, while the F1 score drops from 97.7% to 91.6%. Higher optimization levels result in more aggressive compiler transformations, such as instruction reordering, function inlining, and dead code elimination. These modifications weaken the semantic correspondence between the source code and the generated assembly, thereby reducing the effectiveness of the vulnerability detection tool. Moreover, both Pr and Re decrease simultaneously, indicating that optimization not only reduces the number of detected vulnerabilities (lower recall) but also increases the risk of false positives (lower precision), ultimately leading to a significant decline in the F1 score. These experimental results confirm the negative impact of compilation optimization on vulnerability detection tools. Although the tool performs best at lower optimization levels (e.g., O0), its performance degrades at higher optimization levels.

Table 6 summarizes the comparison of source code-based and assembly code-based detection results. Specifically, in the source code-based analysis method, we no longer perform compilation and disassembly operations, but proceed with the following two phases: (i) slicing and labeling. The source program is first sliced into multiple functions. Based on the four vulnerability candidates (Li et al., 2022), the possible vulnerability sink statement of the training source program is identified. We analyze the parameters of the vulnerability sink statement, and perform the forward slicing using control flow and data flow analysis. All statements that contain these parameters are found, and along with their corresponding line numbers are concatenated in the original order to obtain the final slice, referred to as ScFB. We query the vulnerability location of training sample on the dataset. The line numbers corresponding to the ScFB are iterated, and the ScFB containing the vulnerability line numbers is labeled as “1”, while the ScFB without the vulnerability line numbers is labeled as “0”. (ii) training and detection. After obtaining ScFB, it is fed into the BLSTM-LOC model for training and subsequent vulnerability detection on the test set. It is important to note that, in order to ensure fairness of the results, both analysis methods are conducted on the same dataset, with consistent model architecture, hyperparameters, and training/testing procedures.

It can be found in Table 6 that the model trained from the assembly code-based representation achieves a significant improvement in each

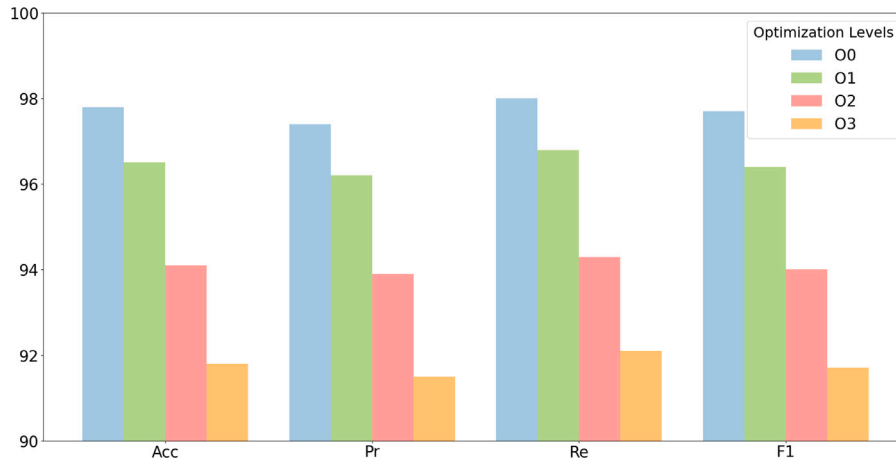


Fig. 11. The impact of different optimization levels on VULOC detection performance.

Table 6

The comparison results of vulnerability detection capability with two different representation methods (unit: %).

Targets	Representations	FPR	FNR	Acc	Pr	Re	F1
Source code	ScFB-based	10.3	7.8	91.5	91.1	92.3	91.9
Assembly code	AcCB-based	1.7	0.2	97.8	97.4	98.0	97.7

metric, relative to the source code-based representation. Especially on *FNR* and *Acc*, it improves by 7.6% and 6.3%, respectively. This can be explained by the fact that the AcCB-based representation approach reduces the fine-grained level of vulnerability detection from the source, since it treats a single line of source code, where the vulnerability originally exists, into multiple lines of assembly code. At the same time, the assembly code holds a simpler syntax structure and a smaller vocabulary than the source code, which facilitates the learning of vulnerability features by neural networks.

Finding 3. The assembly code-based representation approach performs better results than the source code-based representation in vulnerability detection.

5.7. Experiments for answering RQ4

To evaluate the vulnerability detection and localization capabilities of VULOC, we compare the performance with the current state-of-the-art vulnerability detection methods based on two different kinds of datasets.

Table 7 shows the comparison results on the Juliet test suite dataset. Among all the five vulnerability detection methods, VULOC shows the best performance. In particular, there is a 6.6% improvement in *FNR*, the average number of output vulnerability lines improves from 5.2 to 1.0, and the average vulnerability distance decreases to 0.78 lines (the other six methods do not have vulnerability location capability, so the *AD* metric shows “N/A”). Note that we reproduced the VUDDY (Kim et al., 2017) experiments on this dataset and obtained the results, for which it can be interpreted that most of the vulnerabilities are not due to code cloning and therefore VUDDY performs poorly in detecting multiple vulnerabilities. For SySeVR, we also conducted replication experiments on this dataset, which showed that it had an average output line count of 17.1, due to the fact that it is a coarse-grained vulnerability detector based on the program slice level. Since BovdGFE and HAN-BSVD have not released the source code, we replicated the experiments based on the content of the paper and obtained the final experimental results.

Table 7

The comparison of vulnerability detection capability for VULOC and state-of-the-art methods on the Juliet test suite dataset.

Methods	FPR (%)	FNR (%)	Acc (%)	Pr (%)	Re (%)	F1 (%)	AD (line)	VL (line)
RATS	28.7	25.3	70.5	68.9	71.0	69.9	N/A	7.5
Flawfinder	26.6	24.8	76.9	75.1	78.3	76.7	N/A	5.2
VUDDY	6.6	14.5	72.9	70.1	75.7	72.8	N/A	10.4
BovdGFE	3.2	7.8	92.2	93.1	90.0	88.8	N/A	21.5
SySeVR	2.4	6.1	94.2	90.8	94.5	92.6	N/A	17.1
HAN-BSVD	4.5	6.8	96.6	97.9	96.4	97.1	N/A	8.5
VULOC	1.7	0.2	97.8	97.4	98.0	97.7	0.78	1.0

Table 8

The comparison of vulnerability detection capability for VULOC and state-of-the-art methods on the NVD and SARD.

Method	FPR (%)	FNR (%)	Acc (%)	Pr (%)	Re (%)	F1 (%)	AD (line)	VL (line)
Liu et al.	22.4	N/A	83.9	79.7	N/A	84.7	N/A	35.2
Niu et al.	26.6	44.8	69.9	65.1	68.3	66.7	N/A	24.8
BGNN4VD	N/A	N/A	74.7	77.3	76.3	76.8	N/A	18.3
IVDetect	12.1	11.3	78.7	63.3	82.5	71.6	N/A	6.0
LineVul	21.2	39.1	71.4	66.7	60.8	63.6	3.51	1.0
VULOC	10.7	18.3	84.3	83.7	82.5	83.1	1.41	1.0

We apply the t-SNE dimensionality reduction algorithm to visualize the vulnerability detection results of VUDDY, BovdGFE, SySeVR, HAN-BSVD and VULOC in Fig. 12, where “o” indicates non-vulnerable samples and “x” indicates vulnerable samples. The t-SNE can visually represent the separation of vulnerable samples and non-vulnerable samples, and the greater the average distance between them, the higher the vulnerability detection capability. As shown in Fig. 12(e), it can be observed that VULOC achieves highest detection capability because the two kinds of samples are clearly separated and not mixed together.

To more visually demonstrate the vulnerability localization capabilities of VULOC, we plot the location results (as Fig. 13) on the juliet test suite dataset. For the location results curve, the blue line represents the predicted vulnerability line number and the yellow line represents the real vulnerability line number. It is obvious that the more the two curves overlap, the higher the accuracy of VULOC in vulnerability location. We test a total of 3300 vulnerable and non-vulnerable samples, and it can be observed from Fig. 13 that for most of the samples, VULOC correctly predicts the vulnerability location, and only a few have deviations, indicating that VULOC has excellent vulnerability location capabilities.

The experimental results on the NVD dataset are presented in Table 8. It can be observed that VULOC still achieves the optimal performance

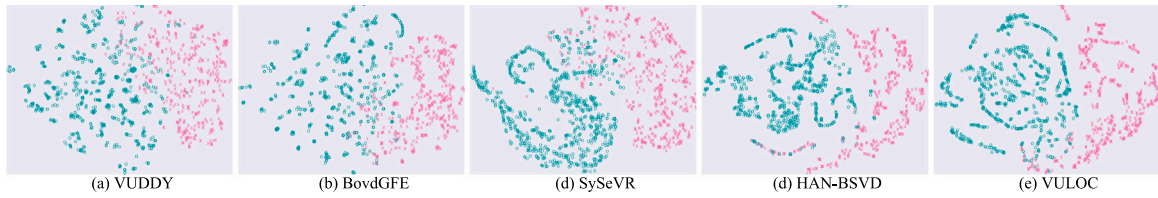


Fig. 12. T-SNE plots illustrating the separation between vulnerable and non-vulnerable examples.

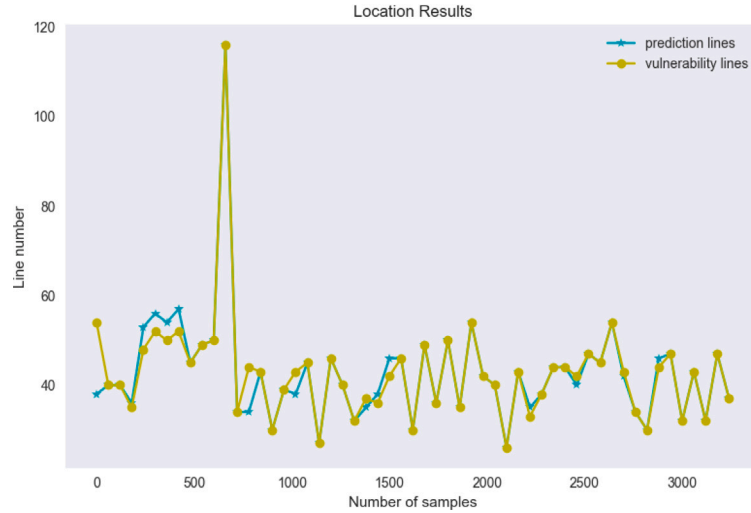


Fig. 13. The location results of VULOC. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

in the NVD, and *AD* decreases to 1.41 lines while having the smallest *VL* of 1.0. Among the other 7 vulnerability detection methods, Liu et al. (2019b) achieve the highest *Acc* and *F1*, however, they hold the highest *VL*, which is due to the fact that its input and output are binary instructions of complete functions, which do not refine the vulnerability detection results and cannot achieve the accurate location of vulnerabilities, resulting in higher *VL* and insufficient granularity of vulnerability detection. In addition, we implemented the LineVul (Fu and Tantithamthavorn, 2022) method on this dataset and obtained experimental results, which showed a significant performance degradation compared to the results reported in their paper, which is due to the fact that they only use the transformer-based model to extract the sequence features of the source code, and many redundant codes not related to vulnerabilities are fed into the model, and the complexity of the vulnerability code also affects its detection ability. Similarly, we replicated the IVDetect (Li et al., 2021b) experiment on this dataset and showed that VULOC was improved by 11.5% on *F1* and 5.6% on *Acc*, respectively, compared to IVDetect.

Finding 4. VULOC is more effective than current state-of-the-art methods in detecting and locating vulnerabilities.

5.8. Experiments for answering RQ5

Despite the outstanding performance achieved by VULOC in the experiments mentioned above, its effectiveness in real-world projects remains unproven. Therefore, we conducted vulnerability tests on 100 source programs from three real software products to validate the vulnerability detection capability of VULOC in practical scenarios.

Table 9 describes the vulnerabilities detected (shown as status “O”) or missed (shown as “×”) by VULOC. In total, VULOC detected 83 vulnerabilities out of the 100 programs that participated in the testing. For these vulnerabilities, which belong to CWE-416, CWE-119, CWE-125,

Table 9

A part of vulnerabilities detected or missed by VULOC in real-world software products.

Software	ID	Vulnerable file	Status
Linux Kernel	CVE-2019-19527	hiddev.c	O
	CVE-2019-19807	timer.c	×
	CVE-2020-12464	message.c	O
	...	...	...
Wireshark	CVE-2018-5334	wiretap_vwr.c	O
	CVE-2019-5718	epan_charsets.c	O
	...	...	...
FFmpeg	CVE-2019-17542	vqavideo.c	O
	CVE-2019-12730	aadec.c	×
	...	...	...

and CWE-787, respectively. These vulnerabilities all involve operations on memory addresses, which can be easily detected based on assembly instruction analysis. For “CVE-2019-19807”, it also belongs to CWE-416 but was not detected by VULOC, indicating that VULOC has some *false-negatives*. In this experiment, we found that VULOC is not sensitive to vulnerabilities spanning multiple functions. This can be attributed to VULOC’s focus on detecting issues within individual functions, rather than emphasizing the connections between multiple functions.

Finding 5. VULOC is still effective to detect vulnerabilities in real-world software products.

6. Limitations

Although the proposed method performs well in vulnerability localization, there are several limitations as follows: (i) the proposed method is only applicable to the vulnerability detection of C/C++

source programs, which cannot be used for other programming languages; (ii) this study required the source program to be compiled into assembly code, therefore, the target program should be successfully compiled, and programs that failed to be compiled cannot be detected; (iii) VULOC takes AcCBs as the detection target, and since AcCBs is function-level based, VULOC cannot detect those vulnerabilities that are caused by cross-functions. (iv) VULOC requires the training dataset to be labeled with the locations of vulnerabilities, otherwise, the model training cannot proceed. (v) Currently, VULOC is only applicable to the assembly code of the x86_64 architecture. In theory, there are certain differences in the assembly code on different architectures. When applying it to other architectures, the neural network needs to be retrained. (vi) Due to the fact that different compiler versions or optimization levels may result in significant variations in the generated assembly code (e.g., optimization options such as O2 and O3 can alter the code structure and the execution order of memory operations), it is therefore necessary to retrain the neural network model accordingly.

7. Usability discussion

In this section, we first discuss the usability of VULOC in real-world software development processes, then analyze the feasibility of porting it to modern compilers, and finally explore its future directions for improvement.

In general, the usual software engineering process is divided into five phases: requirements, design, development, testing, and online. VULOC is compatible with the usual techniques and can be well-suited for both the development and testing phases. For the development phase, VULOC can be designed as a plugin and integrated into the IDE, scanning a c/cpp file whenever the engineer completes it, helping the engineer to find potential loopholes during the writing of the code, and eliminating the subsequent threats and risks caused by the loopholes. During the testing phase, VULOC can be designed as a standalone code security auditing tool. It performs white-box testing on the software by extracting the software source code for scanning. It identifies files and specifies vulnerability locations that may lead to potential vulnerabilities, thereby generating a comprehensive vulnerability detection report. In summary, VULOC can be integrated into conventional software engineering processes, providing a certain level of assurance for the security of software systems.

VULOC was designed with an emphasis on modularity and interface abstraction, ensuring that the core vulnerability localization logic is decoupled from the specific implementations of compiler toolchains (e.g., GCC, GDB, Addr2line). This design choice means that when porting to other modern compilers, such as Clang/LLVM, only adaptations for compiler-specific interfaces and output formats are required, without modifying the core algorithm. Mainstream compilers generally adopt standardized debugging information formats such as DWARF. The primary adaptation efforts focus on the parser component, including adjustments to command-line parameters and output format parsing. Modern compilers may introduce more aggressive optimization strategies, which can alter the structure of debugging information and assembly code, potentially affecting the accuracy of vulnerability localization. To address this, we propose: (i) adding adaptation modules for new optimization strategies, (ii) extending mapping strategies to accommodate more diverse output formats, and (iii) conducting regular testing and validation to ensure high precision and stability in new compiler environments. Considering the above factors, the primary effort in porting the tool lies in adapting interfaces and output parsing, without encountering fundamental technical barriers. The associated maintenance cost and workload remain manageable, allowing for timely adaptation to compiler evolution. Given our modular design and reliance on standardized debugging information, we believe that porting the tool to modern compilers is not a particularly challenging task.

Although VULOC currently supports only C/C++ languages, we plan to add more training data in the future and extend the VULOC method to support additional programming languages, making it more powerful and scalable. Additionally, we will model cross-function vulnerability features to assist VULOC in detecting more complex forms of vulnerabilities. We also intend to integrate source code vulnerability characteristics and incorporate contrastive learning techniques (Gao et al., 2021), with a more refined modeling of vulnerability features to further enhance robustness and generalization ability.

At the same time, considering the platform independence of intermediate code and its more direct representation of the compiler optimization process, as well as the use of Static Single Assignment (SSA) form in LLVM IR (where each variable is assigned only once to simplify code dependencies and enable more efficient use-def chain analysis), leveraging intermediate representations is a promising direction, especially in research environments where they could provide significant advantages. We will consider this option in future work and conduct a more in-depth study of its potential impact.

8. Related work

VULOC combines existing static analysis and deep learning-based vulnerability detection techniques. In contrast to previous static analysis-based approaches, such as the work presented by Zhu et al. (2021) on binary code function modeling for clone detection, which inputs the entire assembly code into the model, thereby introducing a large amount of noise, our proposed method performs fine-grained slicing on the assembly code to eliminate redundant code that may be unrelated to vulnerabilities, improving detection accuracy. Moreover, the vulnerability features extracted by our method possess stronger generalization capabilities, enabling the detection of multiple types of vulnerabilities. Furthermore, in the context of deep learning-based vulnerability detection, we build upon the BLSTM model for binary function vulnerability detection proposed by Liu et al. (2019b) and improve it to develop a specific BLSTM-LOC model capable of vulnerability localization. Similarly, Li et al. (2022), Lv et al. (2023), and others have performed preprocessing on samples, including program slicing, constructing trees or graphs, and so on. We adopt four types of vulnerability candidates proposed by Li et al. (2022) as the vulnerability sink points and perform program slicing on the source code accordingly. Next, we will introduce the related work from both static analysis and deep learning perspectives.

Research on static analysis-based vulnerability detection. Static vulnerability detection is divided into code similarity-based detection and pattern analysis-based detection. There are a number of popular code similarity detection methods in recent years, such as Wang et al. (2020) constructed the FA-AST by augmenting the original AST with explicit control and data flow edges, and then applied two different types of graph neural networks (GNNs) on the FA-AST to measure the similarity of the code pairs. VUDDY (Kim et al., 2017) is a scalable method for vulnerable code clone detection that utilizes function-level granularity and length filtering techniques to reduce the number of signature comparisons and can resist common modifications in cloned code while still preserving vulnerable conditions after applying abstraction techniques, enabling efficient and accurate detection of security vulnerabilities in large software programs. Zhu et al. (2021) proposed a new neural network-based method, Gemini, which computes embeddings, i.e. a numerical vector, based on the control flow graph of each binary function, and then allows effective similarity detection by measuring the distance between the embeddings of two functions. Karanilim (2020) generated the syntax tree of programme code files from which the directly related n-gram structure tokens were extracted and then compared using an information retrieval algorithm, namely cosine correlation in the vector space model. However, since the vast majority of vulnerabilities are not caused by code clones, the code-similarity approach has a high false alarm rate. Pattern analysis-based vulnerability

detection methods can be further divided into traditional analysis tools and machine learning-based detection. Some of the traditional detection tools such as Checkmarx (Ltd., 2023), Coverity (Blackduck, 2024), Fortify (Application, 2023), FlawFinder (Wheeler, 2023), RATS (Secure Software Inc., 2023), etc. require experts to develop rules for detecting vulnerabilities, and their vulnerability detection capabilities are limited. For instance, RATS was originally developed as a rough security auditing tool by Secure Software Inc. It is often used to scan C, C++, etc. source code and flag common security-related programming errors such as buffer overflows, and double free. However, the tool only performs a cursory analysis of the source code, will not find all errors, and will also see what is not an error, which has high positive and negative rates. Coverity analyzes complex logical paths in the code by performing control flow and data flow analysis, along with automated rule matching and symbolic execution techniques, to identify potential defects and vulnerabilities. Recently, Lenarduzzi et al. (2023) assessed six static analysis tools (including Coverity Scan) for their ability to detect source code quality issues. The results indicated that most tools had low accuracy, and the consistency between tools was extremely low, with few tools identifying the same issues. Similarly, Lipp et al. (2022) conducted an empirical study on six static analysis tools (including five open-source and one commercial tool), and their results showed that even the best-performing static code analysis tools missed 47%–80% of real vulnerabilities in various evaluation scenarios. The detection capabilities varied significantly across different types of vulnerabilities. Machine learning based vulnerability detection methods such as Du et al. (2019), Yamaguchi et al. (2012) and Hanif et al. (2021), they detect vulnerabilities at a coarse-grained level such as program level, function level, program slice level and require experts to define vulnerability characteristics.

Research on deep learning-based vulnerability detection. The advantages of deep learning in automatically extracting and learning vulnerability features have led many researchers to apply it to vulnerability detection. Some of the more popular vulnerability detection methods based on deep learning in recent years are described below. As Liu et al. (2019b) encoded the code into binary files, extracted the binary functions by IDA Pro, and then used a Bidirectional Long Short-Term Memory (BLSTM) (Zhou et al., 2016) model to learn and predict vulnerabilities. Li et al. (2020) transformed the source code into the minimal intermediate representation by pre-training the generated corpus, transforming the intermediate representations into vectors, and feeding them into three CNN to learn vulnerability features for vulnerability classification. Wartschinski et al. (2022) encoded the code into vectors by the Word2Vec model and then constructed Long Short-Term Memory (LSTM) networks to classify vulnerable token sequences at a fine-grained level to predict specific regions of the source code that may contain vulnerabilities. Yan et al. (2021) proposed a hierarchical attention network HAN-BSVD for binary software vulnerability detection, which enriches contextual information by a preprocessor with uniform jump addresses and normalized instructions, then preserved contextual information with an instruction embedding network composed of BGRU and word attention modules, and finally, a feature extraction network composed of Text-CNN and spatial attention modules for capturing local features. The multi-granularity vulnerability detector, mVulPreter (Zou et al., 2022), effectively combines the advantages of function-level and slice-level vulnerability detection models and outputs an interpretation of the detection results only through the model itself. Lv et al. (2023) proposed BovdGFE, a vulnerability detection method based on graph feature extraction. They proposed a new code representation structure, called *code representation sequence* (CoRS), which preserves the control flow, data dependencies, and syntactic structure of the vulnerability code in order to reduce the information loss during the code representation process, and then constructed a BGRU+Attention model for vulnerability detection. Li et al. (2022) proposed a deep learning-based SySeVR vulnerability detection framework that obtains program representations that can hold syntactic

and semantic information related to vulnerabilities. SySeVR obtains syntactic-based and semantic-based vector representations through program slicing and encoding and then builds the deep learning model to detect vulnerabilities at the program slice level. Niu et al. (2020) proposed a static taint analysis method based on deep learning to design taint selection rules from the difference files between the source program and its patched program and obtain taint propagation paths through static taint analysis. Finally, a detection model based on BLSTM is applied to discover and locate software vulnerabilities. Cao et al. (2021) proposed a vulnerability detection method based on a Bidirectional Graph Neural Network (BGNN), BGNN4VD, which extracts the syntactic and semantic information of the source code through AST, CFG, and DFG, and then the source code is used as an input to BGNN. Finally, further features are extracted using CNN and vulnerabilities are detected by classifiers.

9. Conclusions

In this paper, we propose a vulnerability localization system based on assembly code slicing, which can effectively detect vulnerabilities in C/C++ programs with high accuracy. VULOC improves the granularity and accuracy of vulnerability detection by performing assembly code slicing techniques and utilizing the mapping relationship between assembly code and source code line numbers and the proposed BLSTM-LOC model. Experiments show that VULOC exhibits excellent performance on both public datasets and remains effective in real-world software products, which provides a new and powerful approach to detect vulnerabilities.

CRedit authorship contribution statement

Xinghang Lv: Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Jianming Fu:** Conceptualization, Funding acquisition, Supervision. **Tao Peng:** Data curation, Formal analysis, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to express their gratitude to people who provided datasets. In addition, the authors would like to thank the anonymous reviewers for their comments and suggestions. This work was funded by the National Natural Science Foundation of China (62272351).

Appendix. Details of the dataset

We present the dataset in this paper in detail through Figs. 14 and 15. First, the original dataset is multiple source programs that contain multiple vulnerable and non-vulnerable self-defined functions. For example, the source program in Fig. 14 contains one vulnerable function: `CWE190_Integer_Overflow_short_rand_square_01_bad()`, three non-vulnerable functions: `goodG2B()`, `goodB2G()`, `CWE190_Integer_Overflow_short_rand_square_01_good()`. The source program was preprocessed by the method in the paper to obtain the structure of Fig. 15 (only two functions from the source program are shown). The sample in Fig. 15 consists of the label (“_lable1_” or “_lable0_”), the compilation path (“File ./testcase/...”), and the AcCB. Further, we processed the dataset in Fig. 14 to obtain the data in Fig. 15 and used them as the training and testing sets for the model.

```

1  /* TEMPLATE GENERATED TESTCASE FILE
2  Filename: CWE190_Integer_Overflow__short_rand_square_01.c
3  Label Definition File: CWE190_Integer_Overflow.label.xml
4  Template File: sources-sinks-01.tmpl.c
5  */
6  /*
7   * @description
8   * CWE: 190 Integer Overflow
9   * BadSource: rand Set data to result of rand()
10  * GoodSource: Set data to a small, non-zero number (two)
11  * Sinks: square
12   *   GoodSink: Ensure there will not be an overflow before squaring data
13   *   BadSink : Square data, which can lead to overflow
14   * Flow Variant: 01 Baseline
15   */
16  */
17
18  #include "std_testcase.h"
19
20  #include <math.h>
21
22  #ifndef OMITBAD
23
24  void CWE190_Integer_Overflow__short_rand_square_01_bad()
25  {
26      short data;
27      data = 0;
28      /* POTENTIAL FLAW: Use a random value */
29      data = (short)RAND32();
30      {
31          /* POTENTIAL FLAW: if (data*data) > SHRT_MAX, this will overflow */
32          short result = data * data;
33          printIntLine(result);
34      }
35  }
36
37  #endif /* OMITBAD */
38
39  #ifndef OMITGOOD
40
41  /* goodG2B uses the GoodSource with the BadSink */
42  static void goodG2B()
43  {
44      short data;
45      data = 0;
46      /* FIX: Use a small, non-zero value that will not cause an overflow */
47      data = 2;
48      {
49          /* POTENTIAL FLAW: if (data*data) > SHRT_MAX, this will overflow */
50          short result = data * data;
51          printIntLine(result);
52      }
53  }
54
55  /* goodB2G uses the BadSource with the GoodSink */
56  static void goodB2G()
57  {
58      short data;
59      data = 0;
60      /* POTENTIAL FLAW: Use a random value */
61      data = (short)RAND32();
62      /* FIX: Add a check to prevent an overflow from occurring */
63      if (abs((long)data) <= (long)sqrt((double)SHRT_MAX))
64      {
65          short result = data * data;
66          printIntLine(result);
67      }
68      else
69      {
70          printLine("data value is too large to perform arithmetic safely.");
71      }
72  }
73
74  void CWE190_Integer_Overflow__short_rand_square_01_good()
75  {
76      goodG2B();
77      goodB2G();
78  }
79
80  #endif /* OMITGOOD */
81
82  /* Below is the main(). It is only used when building this testcase on
83  its own for testing or for building a binary to use in testing binary
84  analysis tools. It is not used when compiling all the testcases as one
85  application, which is how source code analysis tools are tested. */
86
87  #ifdef INCLUDEMAIN
88
89  int main(int argc, char * argv[])
90  {
91      /* seed randomness */
92      srand( (unsigned)time(NULL) );
93
94      #ifndef OMITGOOD
95      printLine("Calling good()...");
96      CWE190_Integer_Overflow__short_rand_square_01_good();
97      printLine("Finished good()");
98      #endif /* OMITGOOD */
99
100     #ifndef OMITBAD
101     printLine("Calling bad()...");
102     CWE190_Integer_Overflow__short_rand_square_01_bad();
103     printLine("Finished bad()");
104     #endif /* OMITBAD */
105     return 0;
106 }
107
108 #endif

```

Fig. 14. A source program in the dataset.

__label__	__label0__
File ./testcases/CWE190_Integer_Overflow/s05/CWE190_Integer_Overflow__short_rand_square_01.c:	File ./testcases/CWE190_Integer_Overflow/s05/CWE190_Integer_Overflow__short_rand_square_01.c:
25: push %rbp	57: push %rbp
25: mov %rsp, %rbp	57: mov %rsp, %rbp
25: push %rbx	57: sub \$0x18, %rsp
25: sub \$0x18, %rsp	59: movw \$0x0, -0x14(%rbp)
27: movw \$0x0, -0x14(%rbp)	61: callq 0x630<rand@plt>
29: callq 0x630<rand@plt>	61: test %eax, %eax
29: and \$0x1, %eax	61: je 0x9ff<goodB2G+51>
29: test %eax, %eax	61: callq 0x630<rand@plt>
29: je 0x9ce<CWE190_Integer_Overflow__short_rand_square_01_bad+51>	61: shl \$0xf, %eax
29: callq 0x630<rand@plt>	61: mov %eax, %ebx
29: callq 0x630<rand@plt>	61: callq 0x630<rand@plt>
29: shl \$0xf, %eax	61: xor %ebx, %eax
29: mov %eax, %ebx	61: jmp 0xa17<goodB2G+75>
29: callq 0x630<rand@plt>	61: callq 0x630<rand@plt>
29: xor %ebx, %eax	61: shl \$0xf, %eax
29: jmp 0x9e6<CWE190_Integer_Overflow__short_rand_square_01_bad+75>	61: mov %eax, %ebx
29: callq 0x630<rand@plt>	61: callq 0x630<rand@plt>
29: callq 0x630<rand@plt>	61: xor %ebx, %eax
29: shl \$0xf, %eax	61: mov %eax, -0x14(%rbp)
29: mov %eax, %ebx	63: cmp \$0xffffffff4b, %eax
29: callq 0x630<rand@plt>	63: jl 0xa4d<goodB2G+129>
29: xor %ebx, %eax	63: movswl -0x14(%rbp), %eax
29: not %eax	63: cmp \$0xb5, %eax
29: mov %eax, -0x14(%rbp)	65: movzwl -0x14(%rbp), %edx
32: movzwl -0x14(%rbp), %edx	65: movzwl -0x14(%rbp), %eax
32: movzwl -0x14(%rbp), %eax	65: mov %eax, -0x12(%rbp)
32: imul %edx, %eax	66: movswl -0x12(%rbp), %eax
32: mov %eax, -0x12(%rbp)	66: mov %eax, %edi
33: movswl -0x12(%rbp), %eax	66: callq 0x79c<printIntLine>
33: mov %eax, %edi	64: jmp 0xa59<goodB2G+141>
33: callq 0x79c<printIntLine>	70: lea 0x12c(%rip), %rdi # 0xb80
35: nop	70: callq 0x75a<printLine>
35: add \$0x18, %rsp	72: add \$0x18, %rsp
35: pop %rbx	72: pop %rbx
35: pop %rbp	72: pop %rbp

Fig. 15. Preprocessed dataset.

Data availability

The source code used in our experiments is available at <https://github.com/DataAvailable/VULOC>.

References

- Application, F., 2023. Fortify. <https://www.microfocus.com/en-us/cyberres/application-security/>.
- Bartlett, J., 2021. Debugging with GDB. In: *Learn To Program with Assembly: Foundational Learning for New Programmers*. Springer, pp. 265–269.
- Blackduck, 2024. Coverity. <https://www.blackduck.com/zh-cn/static-analysis-tools-sast/coverity.html/>.
- Cao, S., Sun, X., Bo, L., Wei, Y., Li, B., 2021. Bgnn4vd: constructing bidirectional graph neural-network for vulnerability detection. *Inf. Softw. Technol.* 136, 106576.
- Center, N., 2019. Juliet test suite for C/C++ 1.3. <https://samate.nist.gov/SARD/test-suites/112>.
- Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2021. Deep learning based vulnerability detection: Are we there yet? *IEEE Trans. Softw. Eng.* 48 (9), 3280–3296.
- Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P., 2002. SMOTE: synthetic minority over-sampling technique. *J. Artificial Intelligence Res.* 16 (1), 321–357.
- Chung, J., Gulcehre, C., Cho, K., Bengio, Y., 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. In: *NIPS 2014 Workshop on Deep Learning*, December 2014.
- Du, X., Chen, B., Li, Y., Guo, J., Zhou, Y., Liu, Y., Jiang, Y., 2019. Leopard: Identifying vulnerable code for vulnerability assessment through program metrics. In: *2019 IEEE/ACM 41st International Conference on Software Engineering*. ICSE, IEEE, pp. 60–71.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al., 2020. CodeBERT: A pre-trained model for programming and natural languages. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. pp. 1536–1547.
- Ferrante, J., Ottenstein, K.J., Warren, J.D., 1987. The program dependence graph and its use in optimization. *ACM Trans. Program. Lang. Syst. (TOPLAS)* 9 (3), 319–349.
- Fu, M., Tantithamthavorn, C., 2022. Linevul: A transformer-based line-level vulnerability prediction. In: *2022 IEEE/ACM 19th International Conference on Mining Software Repositories*. MSR, pp. 608–620.
- Gao, T., Yao, X., Chen, D., 2021. Simcse: Simple contrastive learning of sentence embeddings. In: *EMNLP 2021–2021 Conference on Empirical Methods in Natural Language Processing*, Proceedings.
- GNU, 2023. GCC. <https://gcc.gnu.org/>.
- Grieco, G., Grinblat, G.L., Uzal, L., Rawat, S., Feist, J., Mounier, L., 2016. Toward large-scale vulnerability discovery using machine learning. In: *Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy*. pp. 85–96.
- Grossberg, S., 2013. Recurrent neural networks. *Scholarpedia* 8 (2), 1888.
- Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J., 2022. Unixcoder: Unified cross-modal pre-training for code representation. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. pp. 7212–7225.
- Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Shujie, L., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al., 2020. GraphCodeBERT: Pre-training Code Representations with Data Flow, International Conference on Learning Representations.
- Hanif, H., Nasir, M.H.N.M., Ab Razak, M.F., Firdaus, A., Anuar, N.B., 2021. The rise of software vulnerability: Taxonomy of software vulnerabilities detection and machine learning approaches. *J. Netw. Comput. Appl.* 179, 103009.
- Hardeniya, N., Perkins, J., Chopra, D., Joshi, N., Mathur, I., 2016. *Natural Language Processing: Python and NLTK*. Packt Publishing Ltd.
- Hochreiter, S., 1997. Long short-term memory. *Neural Comput.* MIT- Press.
- Jiang, X., Wang, S., Gong, Y., Yu, T., Liu, L., Yu, X., 2024. HAformer: Semantic fusion of hex machine code and assembly code for cross-architecture binary vulnerability detection. *Comput. Secur.* 145, 104029.
- Karnalim, O., 2020. Syntax trees and information retrieval to improve code similarity detection. In: *Proceedings of the Twenty-Second Australasian Computing Education Conference*. pp. 48–55.
- Kim, S., Woo, S., Lee, H., Oh, H., 2017. Vuddy: A scalable approach for vulnerable code clone discovery. In: *2017 IEEE Symposium on Security and Privacy*. SP, IEEE, pp. 595–614.
- Lenarduzzi, V., Pecorelli, F., Saarimaki, N., Lujan, S., Palomba, F., 2023. A critical comparison on six static analysis tools: Detection, agreement, and precision. *J. Syst. Softw.* 198, 111575.
- Li, Z., Liu, F., Yang, W., Peng, S., Zhou, J., 2021c. A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE Trans. Neural Netw. Learn. Syst.* 33 (12), 6999–7019.
- Li, X., Qu, Y., Yin, H., 2021a. Palmtree: Learning an assembly language model for instruction embedding. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. pp. 3236–3251.
- Li, Y., Wang, S., Nguyen, T.N., 2021b. Vulnerability detection with fine-grained interpretations. In: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. pp. 292–303.
- Li, X., Wang, L., Xin, Y., Yang, Y., Chen, Y., 2020. Automated vulnerability detection in source code using minimum intermediate representation learning. *Appl. Sci.* 10 (5), 1692.
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2022. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* 19 (4), 2244–2258.
- Liang, H., Wang, L., Wu, D., Xu, J., 2016. MLSA: a static bugs analysis tool based on LLVM IR. In: *2016 17th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing*. SNPD, IEEE, pp. 407–412.
- Linux tools, 2019. Addr2line: convert addresses into file names and line numbers. <https://manpages.ubuntu.com/manpages/xenial/en/man1/addr2line>.
- Lipp, S., Banescu, S., Pretschner, A., 2022. An empirical study on the effectiveness of static code analyzers for vulnerability detection. In: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. pp. 544–555.
- Liu, S., Dibaei, M., Tai, Y., Chen, C., Zhang, J., Xiang, Y., 2019b. Cyber vulnerability intelligence for internet of things binary. *IEEE Trans. Ind. Inform.* 16 (3), 2154–2163.
- Liu, J., Yang, Y., Lv, S., Wang, J., Chen, H., 2019a. Attention-based BiGRU-CNN for Chinese question classification. *J. Ambient. Intell. Humaniz. Comput.* 1–12.
- Ltd., C., 2023. Checkmarx. <https://www.checkmarx.com/>.
- Lv, X., Peng, T., Chen, J., Liu, J., Hu, X., He, R., Jiang, M., Cao, W., 2023. BovdGFE: buffer overflow vulnerability detection based on graph feature extraction. *Appl. Intell.* 53 (12), 15204–15221.
- Manès, V.J., Han, H., Han, C., Cha, S.K., Egele, M., Schwartz, E.J., Woo, M., 2019. The art, science, and engineering of fuzzing: A survey. *IEEE Trans. Softw. Eng.* 47 (11), 2312–2331.
- Mani, I., Zhang, I., 2003. kNN approach to unbalanced data distributions: a case study involving information extraction. In: *Proceedings of Workshop on Learning from Imbalanced Datasets*. Vol. 126, ICML, pp. 1–7.
- Niu, W., Zhang, X., Du, X., Zhao, L., Cao, R., Guizani, M., 2020. A deep learning based static taint analysis approach for IoT software vulnerability location. *Meas.* 152, 107139.
- Peng, H., Shoshitaishvili, Y., Payer, M., 2018. T-fuzz: fuzzing by program transformation. In: *2018 IEEE Symposium on Security and Privacy*. SP, IEEE, pp. 697–710.
- Pennington, J., Socher, R., Manning, C.D., 2014. Glove: Global vectors for word representation. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. EMNLP, pp. 1532–1543.
- Rahman, M.M., Ceka, I., Mao, C., Chakraborty, S., Ray, B., Le, W., 2024. Towards causal deep learning for vulnerability detection. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. IEEE, pp. 1–11.
- Samate, 2023. SARD. <https://samate.nist.gov/SARD/>.
- Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M., Monfardini, G., 2008. The graph neural network model. *IEEE Trans. Neural Netw.* 20 (1), 61–80.
- Schuster, M., Paliwal, K.K., 1997. Bidirectional recurrent neural networks. *IEEE Trans. Signal Process.* 45 (11), 2673–2681.
- Secure Software Inc., 2023. RATS. <https://code.google.com/archive/p/rough-auditing-tool-for-security/>.
- She, D., Chen, Y., Shah, A., Ray, B., Jana, S., 2020. Neutaint: Efficient dynamic taint analysis with neural networks. In: *2020 IEEE Symposium on Security and Privacy*. SP, IEEE, pp. 1527–1543.
- US, 2023. NVD. <https://nvd.nist.gov/>.
- Wang, W., Li, G., Ma, B., Xia, X., Jin, Z., 2020. Detecting code clones with graph neural network and flow-augmented abstract syntax tree. In: *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering*. SANER, IEEE, pp. 261–271.
- Wartschinski, L., Noller, Y., Vogel, T., Kehr, T., Grunske, L., 2022. VUDENC: Vulnerability detection with deep learning on a natural codebase for python. *Inf. Softw. Technol.* 144, 106809.
- Wheeler, D.A., 2023. FlawFinder. <http://www.dwheeler.com/flawfinder/>.
- Wong, T.-T., Yeh, P.-Y., 2019. Reliable accuracy estimates from k-fold cross validation. *IEEE Trans. Knowl. Data Eng.* 32 (8), 1586–1594.
- Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., Jin, H., 2022. Vulcnn: An image-inspired scalable vulnerability detection system. In: *Proceedings of the 44th International Conference on Software Engineering*. pp. 2365–2376.
- Yamaguchi, F., Lottmann, M., Rieck, K., 2012. Generalized vulnerability extrapolation using abstract syntax trees. In: *Proceedings of the 28th Annual Computer Security Applications Conference*. pp. 359–368.
- Yan, H., Luo, S., Pan, L., Zhang, Y., 2021. HAN-BSVD: a hierarchical attention network for binary software vulnerability detection. *Comput. Secur.* 108 (13), 102286.
- Yu, Z., Cao, R., Tang, Q., Nie, S., Huang, J., Wu, S., 2020. Order matters: Semantic-aware neural networks for binary code similarity detection. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34, pp. 1145–1152.
- Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Adv. Neural Inf. Process. Syst.* 32.

Zhou, P., Shi, W., Tian, J., Qi, Z., Li, B., Hao, H., Xu, B., 2016. Attention-based bidirectional long short-term memory networks for relation classification. In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers). pp. 207–212.

Zhu, X., Jiang, L., Chen, Z., 2021. Cross-platform binary code similarity detection based on NMT and graph embedding. *Math. Biosci. Eng.* 18 (4), 4528–4551.

Zou, D., Hu, Y., Li, W., Wu, Y., Zhao, H., Jin, H., 2022. MVulPreter: A multi-granularity vulnerability detection system with interpretations. *IEEE Trans. Dependable Secur. Comput.* 1–12.



Xinghang Lv received the master's degree in software engineering from the School of Computer and Artificial Intelligence in 2023, Wuhan Textile University, Wuhan, China. He is currently pursuing his PhD at the School of Cyber Security, Wuhan University. His research areas include software security, vulnerability detection, and artificial intelligence.



Jianming Fu received his B.S. and M.S. degrees from Huazhong University of Science and Technology, and his Ph.D. degree from Wuhan University, where he is currently a professor and Ph.D. supervisor at the School of Cyber Security, Wuhan University. His research interests include vulnerability detection and defense, malicious code detection, and artificial intelligence security.



Tao Peng received the M.Sc. and Ph.D. degrees in computer science from the Huazhong University of Science and Technology in 2006 and 2011, respectively. He is currently an Associate Professor with the School of Computer and Artificial Intelligence, Wuhan Textile University. His research interests include computer vision, pattern recognition, and network security.